« МЮНХЕНСКИЙ УНИВЕРСИТЕТ»

Факультет _____

Кафедра _____

Направление подготовки _____

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

_____

(Фамилия, Имя, Отчество автора)

Тема работы _____

_____

_____

«К защите допущена»

Заведующий кафедрой

ученая степень, звание

……/……

(фамилия , И., О.) / (подпись, МП)

«……»……20…г.

Научный руководитель

ученая степень, звание

должность, место работы

……/……

(фамилия , И., О.) / (подпись, МП)

«……»……20…г.

Дата защиты: «……»……20…г.

МЮНХЕН, 20__

Аннотация

Работа посвящена разработке новых эвристических методов планирования запросов в реляционных СУБД. В литературе известно достаточно много алгоритмов планирования, но в индустриальных применениях известны лишь базовые подходы, опирающиеся на динамическое программирование, жадные и генетические алгоритмы и их простейшие эвристические комбинации, позволяющие выбирать тот или иной алгоритм в зависимости от числа таблиц в запросе. Данная работа преследует цель получить глубокое понимание того, как разные алгоритмы возможно комбинировать в ходе планирования запроса, делая переключение между алгоритмами в зависимости от топологии и сложности графа запроса. Ожидаемым результатом работы является решение оптимизационной задачи планирования с помощью новых эвристик, реализация решения в виде модификации планировщика в ядре реляционной СУБД с открытым кодом и экспериментальная оценка на известных индустриальных бенчмарках.

Содержание

1. Введение **5**

2. Формальная постановка задачи **14**

3. Предшествующие работы **16**

3.1. Задача выбора порядка соединений 16

3.2. Принадлежность NP 16

3.3. NP-полнота 17

3.4. Пространство поиска планов 17

3.5. Жадный подход к планированию 18

3.6. Динамическое программирование по размеру подпланов (DPsize) 20

3.7. Динамическое программирование по подмножествам (DPsub) 23

3.8. Динамическое программирование по парам связных подграфов (DPccp) 26

3.9. DPhyp: динамическое программирование на гиперграфах . . 29

3.9.1. Гиперграф и csg-cmp-пары 29

3.9.2. Идея перечисления 30

3.9.3. DPhyp 31

3.10. Генетический подход 32

3.11. Использование графовых топологий при оценке эвристик . 34

3.12. Адаптивная оптимизация очень больших запросов соединения 36

3.13. Архитектура планировщика PostgreSQL 38

3.14. Оценки в модели стоимости 41
3.15. Открытые вопросы 46

4. Основная часть 47

4.1. Окружение 47
4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения 50
4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии 55
4.4. Подход 3. Динамическое программирование по подмножествам (DPsub) 62
4.5. Подход 4. DPhyp (реализация с открытым исходным кодом) 64
4.6. Подход 5. Модифицированный DPhyp (DPhypMod) 67
4.7. Подход 6. Контракция якорных зон 72

5. Ограничения и применимость результатов 79

1. Введение

Реляционная СУБД хранит данные в виде таблиц (отношений), состоящих из строк и столбцов. Для формулирования запросов к данным используется декларативный язык SQL. С ростом объёма данных и сложности запросов увеличивается время, необходимое для их выполнения, и сокращение этого времени становится важным фактором эффективности СУБД.

Язык SQL. Стандартным средством взаимодействия с реляционными СУБД является язык SQL (Structured Query Language) — декларативный язык запросов, в котором пользователь описывает *какие* данные необходимо получить, но не указывает *как* именно их извлечь. Типичный запрос на чтение данных имеет вид `SELECT … FROM … WHERE …` и задаёт: список возвращаемых атрибутов (проекция), набор таблиц-источников и условия соединения между ними, а также фильтрующие условия на строки. Один и тот же результат может быть получен множеством различных способов исполнения; выбор конкретного способа — задача оптимизатора СУБД.

В процессе трансляции запрос превращается сначала в логическое представление в виде дерева, затем с помощью оптимизатора СУБД в физический план исполнения. Производительность СУБД напрямую зависит от качества построенного физического плана. Для создания ”хорошего” плана нужно решить или приблизить решение NP-полной задачи выбора оптимального порядка соединений таблиц, так как оно требует сложных вычислений и значительных затрат ресурсов операционной системы. В процессе работы оптимизатора, решается вопрос какой тип соединения использовать, как и в каком порядке соединить таблицы. Запрос преобразу-

ется в набор планов.

Соединение таблиц – это операция, которая позволяет объединять данные из двух и более таблиц по определённому условию. Использование соединений необходимо, когда информация распределена между несколькими таблицами. Соединение принимает два аргумента, назовём первый аргумент левым, второй правым. Каждый аргумент это таблица, либо результат соединения нескольких таблиц.

Отношение – либо единичная таблица, либо результат соединения нескольких таблиц.

Кардинальность отношения R — это число строк в R, обозначается $|R|$. Для базовой таблицы кардинальность известна из системного каталога. Для промежуточного результата $R_i \bowtie R_j$ кардинальность неизвестна заранее и оценивается оптимизатором на основе статистик; обозначим оценку $|\widehat{R_i \bowtie R_j}|$.

Селективность предиката соединения p — это доля строк декартова произведения $R_i \times R_j$, удовлетворяющих условию p:

$$\mathrm{sel}(p) \;=\; \frac{|R_i \bowtie_p R_j|}{|R_i| \cdot |R_j|}, \qquad 0 \leqslant \mathrm{sel}(p) \leqslant 1.$$

Кардинальность результата соединения выражается через селективность: $|R_i \bowtie_p R_j| = \mathrm{sel}(p) \cdot |R_i| \cdot |R_j|$. Оптимизатор оценивает $\mathrm{sel}(p)$ по статистикам (гистограммам, списку наиболее частых значений), предполагая независимость атрибутов. При нарушении этого предположения ошибки оценок селективности каскадируются мультипликативно по цепочке соединений.

Виды соединений таблиц:

1. **Внутреннее соединение** - возвращает только те строки, для которых условие связи выполняется в обоих аргументах.
2. **Внешнее соединение** - возвращает все строки из одного или обоих аргументов, дополняющее отсутствующие совпадения пустыми значениями. Существует три вида внешних соединений.
   (a) **Левое внешнее соединение** - возвращает все строки из левого аргумента и совпадения из правого.
   (b) **Правое внешнее соединение** - возвращает все строки из правого аргумента и совпадения из левого.
   (c) **Полное внешнее соединение** - возвращает объединение всех строк из обоих аргументов.
3. **Полусоединение** - возвращает строки из левого аргумента по существованию хотя бы одного совпадения в правом аргументе.
4. **Антисоединение** - это операция, обратная полусоединению, которая возвращает строки из левого аргумента только при отсутствии совпадений в правом.
5. **Декартово произведение** - соединяет каждую строку левого аргумента с каждой строкой правого. Выполняет соединение без условия.

Стадии обработки запроса. Путь от текстового SQL-запроса до получения результата в PostgreSQL проходит через несколько последовательных стадий:

1. **Лексический и синтаксический анализ.** Текст запроса разбирается на токены и преобразуется в дерево разбора — синтаксическое

представление запроса, отражающее его грамматическую структуру. На этом этапе проверяется корректность синтаксиса, но семантика запроса ещё не анализируется.

2. **Семантический анализ.** Дерево разбора обогащается информацией из системного каталога: разрешаются имена таблиц и атрибутов, проверяются типы данных, раскрываются псевдонимы и представления (views). Результат — дерево запроса (query tree), в котором все объекты идентифицированы однозначно.

3. **Переписывание.** К дереву запроса применяются правила переписывания: раскрытие представлений, применение правил безопасности на уровне строк, преобразование подзапросов. Семантика запроса сохраняется, но его структура может существенно измениться.

4. **Логическая оптимизация.** Переписанное дерево преобразуется в логическое представление: определяется набор базовых отношений, извлекаются предикаты соединения и фильтрующие условия, строятся классы эквивалентности атрибутов. На этом этапе формируется граф соединений $G = (\mathcal{R}, \mathcal{E})$, который является входом для алгоритмов выбора порядка соединений.

5. **Физическая оптимизация (планирование).** Для графа соединений выбирается порядок соединения отношений, для каждого соединения определяется физический алгоритм исполнения (вложенный цикл, хеш-соединение, соединение слиянием), а также способы доступа к данным (последовательное сканирование, сканирование по индексу). Результат — физический план исполнения, представляющий собой де-

рево операторов с конкретными алгоритмами в каждом узле. Именно на этой стадии решается задача выбора порядка соединений, которой посвящена данная работа.

6. **Исполнение.** Физический план передаётся исполнителю, который обходит дерево операторов и формирует результат запроса.

Таким образом, задача оптимизации порядка соединений возникает на стадии физической оптимизации, после того как из исходного SQL-запроса извлечены базовые отношения $\mathcal{R}$, предикаты соединения $\mathcal{P}$ и построен граф соединений $\boldsymbol{G}$.

Оптимизатор СУБД использует стоимостную модель, которая преобразует прогнозы использования различных ресурсов в одно число. **Стоимость**. Ключевые компоненты, которые учитываются, это:

1. Затраты на ввод-вывод (I/O Cost): Чтение/запись данных с диска.
2. Затраты на ЦПУ (CPU Cost): Обработка данных (сравнения, вычисления, агрегация).
3. Затраты на передачу данных (Network Cost): Передача данных между узлами (в распределённых системах).
4. Затраты на память (Memory Cost): Использование оперативной памяти для временных структур.
5. Затраты на конкурентность (Concurrency Cost): Влияние на блокировки и параллельный доступ.

Каждый план можно представить в виде дерева, вершинами в котором являются отношения. Рёбра — условия соединения отношений. Операция внутреннего соединения коммутативна и ассоциативна по результату:

$(R_1 \bowtie R_2) \bowtie R_3$ и $R_1 \bowtie (R_2 \bowtie R_3)$ возвращают одно и то же множество строк. Однако *стоимость вычисления* этих выражений, как правило, различна: $\mathrm{Cost}\big((R_1 \bowtie R_2) \bowtie R_3\big) \neq \mathrm{Cost}\big(R_1 \bowtie (R_2 \bowtie R_3)\big)$, поскольку кардинальности промежуточных результатов зависят от порядка. Внешние соединения (outer join) не ассоциативны и не коммутативны уже на уровне семантики: перестановка аргументов может изменить результат. Это дополнительно ограничивает множество допустимых порядков соединений.

Выбор, в какой последовательности нужно соединить таблицы, является задачей выбора порядка соединений. Различные планы исполнения для одного и того же запроса возвращают одинаковый результат, но время и ресурсы (CPU, память, I/O обращения, возможно сетевые ресурсы), необходимые для выполнения запроса, сильно различаются. Поэтому выбор оптимального плана позволяет сократить время отклика, минимизировать потребление ресурсов и эффективно обрабатывать большие массивы данных, значительно повышая удобство работы пользователя.

Комбинаторная природа задачи видна через простую оценку. Если соединения выполнять бинарно, план задаётся полным двоичным деревом с n листьями (таблицами). Число различных форм таких деревьев — C_{n-1}, где C_k — число Каталана. С учётом перестановок таблиц по листьям число различных планов составляет $C_{n-1} \cdot n!$; асимптотически $C_{n-1} \cdot n! \sim \dfrac{4^{n-1} \cdot n!}{n^{3/2}\sqrt{\pi}}$. Даже для $n = 10$ это $\approx 1{,}7 \cdot 10^{10}$ планов. Это экспоненциальный рост количества возможных планов.

В работе запрос рассматривается как (гипер)граф соединений. Вершины — отношения (таблицы), рёбра — условия соединений. Рёбра, затрагивающие более двух таблиц – гипер-рёбра. Ситуацию осложняют внешние

соединения (`Outer join`): в общем случае они семантически не ассоциативны и не коммутативны, следовательно, ограничивают допустимые переупорядочивания.

Можно заметить, что гиперграф запроса можно разбить на различные топологии: цепи, циклы, звёзды, плотные графы. Количество возможных планов у двух топологий (с обычными рёбрами и без внешних соединений) от одинакового числа таблиц может отличаться в несколько раз, при увеличении количества таблиц разница может вырасти до нескольких порядков.

На практике индустриальные СУБД комбинируют подходы. Для малого числа таблиц применяют динамическое программирование (перебор подмножеств с запоминанием лучших частичных планов). При росте количества таблиц переключаются на эвристики: жадные стратегии, локальные улучшения, генетические алгоритмы и их простые сочетания. У такого подхода есть существенный недостаток: количество возможных планов коррелирует с числом таблиц, но линейная зависимость отсутствует.

В данной работе предлагаются методы адаптивного планирования запросов, которые включают в себя:

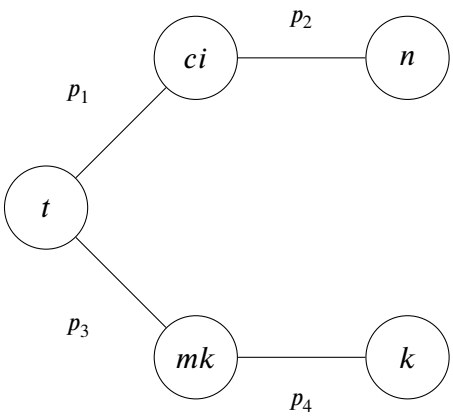
1. Различные эвристики для планирования запроса.
2. Критерии для выбора эвристик для планирования запроса.
3. Критерии для динамического переключения между эвристиками во время планирования.
4. Реализацию динамическое переключение между эвристиками в зависимости от критериев.

Сквозной пример. Рассмотрим SQL-запрос, который находит названия

фильмов жанра «комедия», вышедших после 2010 года, вместе с именами актёров:

```
SELECT t.title, n.name
FROM   title       AS t,
       cast_info   AS ci,
       name        AS n,
       movie_keyword AS mk,
       keyword     AS k
WHERE  ci.movie_id    = t.id
  AND  ci.person_id   = n.id
  AND  mk.movie_id    = t.id
  AND  mk.keyword_id  = k.id
  AND  k.keyword      = 'comedy'
  AND  t.production_year > 2010;
```

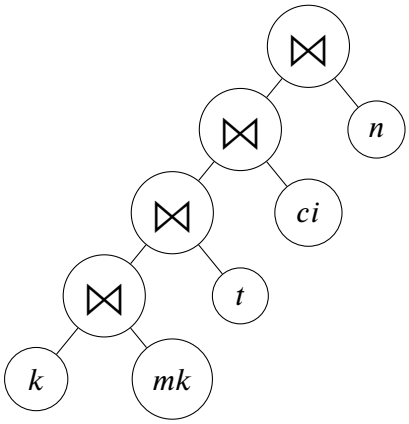
После стадий разбора, семантического анализа и логической оптимизации планировщик извлекает пять базовых отношений $\mathcal{R} = \{t,\ ci,\ n,\ mk,\ k\}$ и четыре предиката соединения, образующие граф соединений G:



где $p_1 : ci.movie_id = t.id$, $p_2 : ci.person_id = n.id$, $p_3 : mk.movie_id = t.id$, $p_4 : mk.keyword_id = k.id$. Фильтрующие условия $k.keyword =$

‘comedy’ и $t.production_year > 2010$ применяются локально к таблицам k и t. Граф имеет топологию *звезда с центром* t и двумя лучами: $n - ci - t$ и $k - mk - t$.

На стадии физической оптимизации планировщик перебирает порядки соединений. Один из возможных планов — левосвязанное дерево:



$$T_1 = (((k \bowtie mk) \bowtie t) \bowtie ci) \bowtie n$$

Этот план начинает с малой таблицы k (после фильтра $keyword =$ ‘comedy’ остаётся одна строка), соединяет с mk по индексу, затем присоединяет t, ci и n. Промежуточные результаты растут постепенно, что минимизирует объём обработки. Альтернативный порядок, в котором сначала соединяются большие таблицы ci и t (кардинальности ~36 млн и ~2,5 млн), породил бы промежуточный результат в миллионы строк уже на первом шаге, многократно увеличив время исполнения.

Данный пример показывает, как структура графа соединений, селективность предикатов и порядок соединений совместно определяют эффективность плана исполнения.

2. Формальная постановка задачи

Пусть задан запрос, затрагивающий n базовых отношений $\mathcal{R} = \{R_1, \dots, R_n\}$, и множество предикатов соединения $\mathcal{P}$, определяющее граф соединений $G = (\mathcal{R}, \mathcal{E})$, где ребро $(R_i, R_j) \in \mathcal{E}$ существует тогда и только тогда, когда имеется предикат, связывающий R_i и R_j.

План исполнения запроса — это полное бинарное дерево T с n листьями, помеченными отношениями из $\mathcal{R}$, в котором каждый внутренний узел задаёт операцию соединения своих двух поддеревьев. Обозначим $\mathcal{T}_n$ — множество всех допустимых планов для данного запроса; $|\mathcal{T}_n|$ растёт как $C_{n-1} \cdot n!$, где C_k — число Каталана ($C_k \sim \frac{4^k}{k^{3/2}\sqrt{\pi}}$).

Стоимостная модель. СУБД использует функцию стоимости $\mathrm{Cost}: \mathcal{T}_n \to \mathbb{R}_{\geqslant 0}$, которая по статистикам отношений (оценкам кардинальностей, селективностей предикатов, доступности индексов) присваивает каждому плану T скалярную оценку ресурсозатрат. В идеальном случае $\mathrm{Cost}(T)$ монотонно коррелирует с фактическим временем исполнения плана T; на практике эта корреляция нарушается из-за ошибок оценки кардинальностей промежуточных результатов [1].

Алгоритм планирования. Алгоритм планирования — это отображение $\mathcal{A}: (\mathcal{R}, \mathcal{P}, \Sigma) \to \mathcal{T}_n$, где Σ — множество доступных статистик (гистограммы, MCV-списки, оценки n_{distinct} для каждого столбца). Результат $T_{\mathcal{A}} = \mathcal{A}(\mathcal{R}, \mathcal{P}, \Sigma) \in \mathcal{T}_n$ — план исполнения. Работа алгоритма $\mathcal{A}$ характеризуется двумя величинами:

- $t_{\text{plan}}(\mathcal{A})$ — время работы алгоритма планирования (время перебора, оценки и выбора плана);

- $t_{\text{exec}}(\mathcal{A})$ — время исполнения выбранного плана $T_{\mathcal{A}}$ на данных.

Время исполнения зависит от качества выбранного плана: чем ближе $T_{\mathcal{A}}$ к истинному оптимуму по фактическому времени, тем меньше t_{exec}. Время планирования определяется алгоритмом $\mathcal{A}$: точные методы (полный DP-перебор) дают малое t_{exec}, но большое t_{plan} при росте n; быстрые эвристики сокращают t_{plan}, но могут увеличить t_{exec}.

Целевая функция. Суммарное время обработки запроса:

$$t_{\text{total}}(\mathcal{A}) \;=\; t_{\text{plan}}(\mathcal{A}) \;+\; t_{\text{exec}}(\mathcal{A}) \;\longrightarrow\; \min_{\mathcal{A}}.$$

Задача состоит в построении алгоритма планирования $\mathcal{A}^*$, минимизирующего t_{total} на заданном классе запросов $\mathcal{Q}$:

$$\mathcal{A}^* \;=\; \arg\min_{\mathcal{A}} \sum_{q \in \mathcal{Q}} t_{\text{total}}(\mathcal{A},\, q).$$

В данной работе в качестве $\mathcal{Q}$ используется Join Order Benchmark (JOB) — 113 аналитических запросов с 5–17 соединениями к базе данных IMDb [1].

Конфликт планирования и исполнения. Задача минимизации t_{total} принципиально отличается от классической задачи минимизации стоимости плана $\min_{T \in \mathcal{T}_n} \text{Cost}(T)$, поскольку включает время самого поиска в $\mathcal{T}_n$. При малом n точный перебор (DP) решает обе подзадачи: t_{plan} пренебрежимо, а t_{exec} минимально. При большом n возникает компромисс: полный перебор $\mathcal{T}_n$ экспоненциален, и t_{plan} начинает доминировать. Необходим *адаптивный* подход, выбирающий стратегию планирования в зависимости от структуры графа G и доступного бюджета времени.

3. Предшествующие работы

3.1. Задача выбора порядка соединений

В статье Ibaraki & Kameda [2] формализуется задача выбора оптимального порядка соединения n таблиц, при использовании вложенного цикла для сканирования таблиц и результатов соединений. **Оптимальный вложенный порядок**. Для заданного набора отношений и предикатов соединения выводится формула ожидаемого числа чтений страниц как функция от порядка соединяемых отношений, и требуется найти порядок, минимизирующий это значение. Авторы прямо отмечают, что задача нахождения минимального значения этой функции NP-полная.

Задача выбора оптимального вложенного порядка принимает на вход:

1. Статистики для формулы ожидаемого числа чтений страниц.
2. Описание запроса (граф соединений).
3. Пороговое значение B — существует ли такой порядок вложенности отношений, что число чтений страниц $\leq B$?

3.2. Принадлежность NP

Если мы угадали порядок отношений π, то значение функции числа чтений страниц для π вычисляется полиномиально по размеру входа (это просто вычисление выражения по заданным параметрам). Значит, решение можно проверить за полиномиальное время, тогда задача принадлежит NP.

3.3. NP-полнота

В доказательстве NP-полноты авторы сводят к этой задаче классическую NP-полную задачу поиска полного графа [3]: по графу $G = (V, E)$ и числу k строится экземпляр запроса (в терминах отношений и условий соединения) и подбираются параметры стоимости так, что существует порядок с числом чтений страниц $\leq B$ тогда и только тогда, когда в G есть полный граф размера k.

3.4. Пространство поиска планов

Пусть даны два отношения R1 и R2, и дан результат их соединения ($R1 \bowtie R2$). Назовём R1 - левым сыном, R2 - правым сыном, а результат соединения родителем. Тогда получится двоичное дерево. Каждому возможному порядку соединения исходного набора таблиц сопоставим двоичное дерево.

Для заданного начального дерева **пространство поиска планов** [4] — это множество всех различных порядков, которые можно получить, применяя к начальному плану последовательность из корректных преобразований (они сохраняют эквивалентность результатов, т.е семантическую корректность).

Виды корректных преобразований:

1. **Коммутативное**: $(R \bowtie_{RS} S) = (S \bowtie_{SR} R)$
2. **Ассоциативное**: $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = R \bowtie_{RS,RT} (S \bowtie_{ST} T)$
3. **Левоассоциативное**: $(R \bowtie_{RS} S) \bowtie_{RT,ST} T = (R \bowtie_{RT} T) \bowtie_{RS,ST} S$

4. **Правоассоциативное**: $R \bowtie_{RS,RT} (S \bowtie_{ST} T) = S \bowtie_{RS,ST} (R \bowtie_{RT} T)$

3.5. Жадный подход к планированию

В статье [5] предлагается эвристический способ построения оптимального порядка, путём ухода от экспоненциального перебора порядков соединений, строя порядок соединений жадно снизу-вверх:

1. На каждом шаге выбирать такое соединение, которое имеет минимальную стоимость промежуточного результата, с учётом селективностей предикатов.
2. Соединять выбранные поддеревья.
3. Обновлять граф запроса новыми оценками размеров/селективностей.

Algorithm 1 GOO($\{R_1, \ldots, R_n\}$, $C(T_1, T_2)$)

```
Input: set of relations to be joined
Input: join tree
Trees ← {R_1, ..., R_n}
while |Trees| ≠ 1 do
    find T_i, T_j ∈ Trees such that i ≠ j and C(T_i, T_j) is minimal among all pairs of trees in Trees
    Trees ← Trees ∖ {T_i}
    Trees ← Trees ∖ {T_j}
    Trees ← Trees ∪ {T_i ⋈ T_j}
end while
return the (only) tree contained in Trees
```

Преимущества:

1. Эвристика строит ветвистые деревья, что позволяет параллельно исполнять дерево плана.
2. Полиномиальность по времени планирования $O(n^3)$, где n — число начальных отношений, вместо экспоненциального времени у полного перебора.

Недостатки:

1. Нет гарантий оптимальности: локально лучший шаг может загнать в глобально плохой порядок, классическая проблема жадных алгоритмов.
2. Зависимость от качества оценок кардинальностей/селективностей: если оценки ошибочны, жадный подход резко деградирует.

В данной статье есть связь с работой [2]. Так как в общем виде задача выбора оптимального порядка соединения отношений NP-полная, то планирование больших запросов требует больших вычислительных ресурсов, следовательно, нужны эвристики.

В современных СУБД самый распостранённый подход к планированию это **динамическое программирование**. Алгоритмы динамического программирования — это методы решения задач оптимизации, которые разбивают исходную задачу на набор перекрывающихся подзадач и используют принцип оптимальности: оптимальное решение строится из оптимальных решений подзадач. Вместо повторного пересчёта результатов они их запоминают, снижая асимптотическую сложность по сравнению с полным перебором. Такие алгоритмы особенно эффективны, когда пространство решений можно параметризовать состояниями (например, подмножества-

ми, интервалами, путями в графе). Цена за ускорение — рост памяти и экспоненциальное число состояний в худшем случае.

3.6. Динамическое программирование по размеру подпланов (DPsize)

В пособии [6] алгоритм DPsize рассматривается как вариант динамического программирования для построения оптимального ветвистого дерева соединений при условии, что граф соединений запроса связный. Алгоритм перечисляет планы в порядке возрастания размера подпланов (числа отношений внутри подплана).

DPsize поддерживает таблицу $BestPlan(S)$, сопоставляющую каждому множеству отношений S лучший (минимальной стоимости) найденный план, и строит планы снизу-вверх:

1. Инициализация: для каждого отношения R_i задаётся $BestPlan(\{R_i\}) = R_i$.
2. Для размера плана $s = 2, \dots, n$ (по возрастанию) перебираются разбиения $s = s_1 + s_2$, где $1 \leq s_1 \leq \lfloor s/2 \rfloor$.
3. Для всех множеств S_1, S_2, уже имеющихся в $BestPlan$, таких что $|S_1| = s_1$, $|S_2| = s_2$, проверяется:
   (a) $S_1 \cap S_2 = \varnothing$ (подпланы не перекрываются),
   (b) S_1 соединено с S_2 хотя бы одним предикатом.
4. Если проверки пройдены, строится кандидат $CurrPlan = CreateJoinTree(BestPlan(S_1), BestPlan(S_2))$ и обновляется

$BestPlan(S_1 \cup S_2)$, если кандидат дешевле.

Algorithm 2 DPsize($\{R_1, \dots, R_n\}$, $C(T_1, T_2)$)

1: **Input:** A set of relations $R = \{R_1, \dots, R_n\}$ to be joined
2: **Output:** An optimal bushy join tree
3: $B \leftarrow$ an empty DP table $2^R \rightarrow$ join tree
4: **for each** $R_i \in R$ **do**
5: $B[\{R_i\}] \leftarrow R_i$
6: **end for**
7: **for each** $1 < s \leq n$ **ascending do**
8: **for each** $S_1, S_2 \subset R$ **such that** $|S_1| + |S_2| = s$ **do**
9: **if** (**not** cross products $\wedge \neg S_1$ connected to S_2) $\vee$ ($S_1 \cap S_2 \neq \varnothing$) **then**
10: **continue**
11: **end if**
12: $p_1 \leftarrow B[S_1], p_2 \leftarrow B[S_2]$
13: **if** $p_1 = \epsilon$ **or** $p_2 = \epsilon$ **then continue**
14: **end if**
15: $P \leftarrow$ CreateJoinTree(p_1, p_2)
16: **if** $B[S_1 \cup S_2] = \epsilon$ **or** $C(B[S_1 \cup S_2]) > C(P)$ **then**
17: $B[S_1 \cup S_2] \leftarrow P$
18: **end if**
19: **end for**
20: **end for**

Преимущества:

1. Оптимальность (в рамках выбранной стоимостной модели и связных графов запросов): DPsize перебирает все допустимые соединения

подпланов и сохраняет лучший результат для каждого множества S.

2. Структурированное перечисление снизу-вверх по размеру подпланов удобно для реализации.

3. Зависимость времени от топологии: для цепей и циклов автор приводит полиномиальные формулы числа внутренних проверок ($O(n)$), что объясняет практическую применимость DPsize на простых топологиях.

Недостатки:

1. Худший случай экспоненциален: для звезды и полного графа число комбинаций резко возрастает.

2. Далёк от нижней границы по перебору: в книге подчёркнуто, что DPsize (как и DPsub) перебирает существенно больше, чем необходимый минимум, что и мотивирует использовать DPccp.

Сложность DPsize от числа отношений n:

$$I_{\text{DPsize}}^{\text{chain}}(n) = \begin{cases} \frac{1}{48}(5n^4 + 6n^3 - 14n^2 - 12n), & n \text{ even} \\ \frac{1}{48}(5n^4 + 6n^3 - 14n^2 - 6n + 11), & n \text{ odd} \end{cases}$$

$$I_{\text{DPsize}}^{\text{cycle}}(n) = \begin{cases} \frac{1}{4}(n^4 - n^3 - n^2), & n \text{ even} \\ \frac{1}{4}(n^4 - n^3 - n^2 + n), & n \text{ odd} \end{cases}$$

$$I_{\text{DPsize}}^{\text{star}}(n) = \begin{cases} 2^{2n-4} - \frac{1}{4}\binom{2n}{n-1} + q(n), & n \text{ even} \\ 2^{2n-4} - \frac{1}{4}\binom{2(n-1)}{(n-1)} + \frac{1}{4}\binom{(n-1)}{(n-1)/2} + q(n), & n \text{ odd} \end{cases}$$

$$\text{with } q(n) = n2^{2n-1} - 5 \times 2^{n-3} + \frac{1}{2}(2^n - 5n + 4)$$

$$I_{\text{DPsize}}^{\text{clique}}(n) = \begin{cases} 2^{2n-2} - 5 \times 2^{n-2} + \frac{1}{4}\binom{2n}{n} - \frac{1}{4}\binom{n}{n/2} + 1, & n \text{ even} \\ 2^{2n-2} - 5 \times 2^{n-2} + \frac{1}{4}\binom{2n}{n} + 1, & n \text{ odd} \end{cases}$$

3.7. Динамическое программирование по подмножествам (DPsub)

В [6] описывается также вариант динамического программирования для построения оптимального ветвистого дерева соединений для связного графа запроса. В отличие от DPsize, где планы строятся по возрастанию размера подпланов, DPsub перебирает все непустые подмножества исходного множества таблиц и для каждого подмножества строит лучший план.

```
Input: A set of relations R = {R_1, ..., R_n} to be joined
Output: An optimal bushy join tree
B ← an empty DP table 2^R → join tree
for each R_i ∈ R do
    B[{R_i}] ← R_i
end for
for each 1 < i ≤ 2^n − 1 ascending do
    S ← {R_j ∈ R | (⌊i/2^(j−1)⌋ mod 2) = 1}
    for each S_1, S_2 ⊂ S such that S_2 = S ∖ S_1 do
        if (not cross products ∧¬S_1 connected to S_2) then
            continue
        end if
        p_1 ← B[S_1], p_2 ← B[S_2]
        if p_1 = ϵ or p_2 = ϵ then continue
        end if
        P ← CreateJoinTree(p_1, p_2)
        if B[S] = ϵ or C(B[S]) > C(P) then
            B[S] ← P
        end if
    end for
end for
```

Преимущества:

1. Эффективен на плотных пространствах поиска(звезда/полный граф): в таких графах больше связных подмножеств и больше разбиений

$S = S_1 \cup S_2$, проходящих проверки, поэтому доля холостых итераций меньше; на этих топологиях DPsub начинает выигрывать у DPsize.

2. Оптимальность (в рамках выбранной стоимостной модели и связных графов запросов): DPsub перебирает все допустимые разбиения S на две части и сохраняет лучший план для каждого S.

Недостатки:

1. Большое количество холостых итераций для простых топологий (цепь/цикл): DPsub перебирает все подмножества S, но значительная часть из них несвязна, а для связных S большая доля разбиений (S_1, S_2) не проходит проверки связности для соединения.

2. Далёк от теоретической нижней границы: для большинства топологий число проверок во внутреннем цикле на порядки больше числа количества пар связных подграфов, что служит мотивацией перехода к DPccp.

Сложность DPsub от числа отношений n:

$$I_{\text{DPsub}}^{\text{chain}}(n) = 2^{n+2} - n^2 - 3n - 4$$
$$I_{\text{DPsub}}^{\text{cycle}}(n) = n2^n + 2^n - 2n^2 - 2$$
$$I_{\text{DPsub}}^{\text{star}}(n) = 2 \times 3^{n-1} - 2^n$$
$$I_{\text{DPsub}}^{\text{clique}}(n) = 3^n - 2^{n+1} + 1$$

3.8. Динамическое программирование по парам связных подграфов (DPccp)

Пусть дан граф соединений $G = (V, E)$. Введём понятие csg-cmp-pair (S_1, S_2) — в графе запроса выделим S_1, S_2 — связные непересекающиеся подграфы, такие что $S_1 \subseteq V$, $S_2 \subseteq V \setminus S_1$ (отсутствие пересечения) и существует между ними ребро. S_1, S_2 в (S_1, S_2) называются комплементарной парой.

Определим #csg — количество связных подграфов, в определении csg-cmp-pair это S_1 или S_2.

Определим #ccp — количество csg-cmp-pairs.

В [7] предлагается алгоритм DPccp как улучшение DPsize/DPsub для построения оптимального ветвистого дерева для связного графа запроса и получения нижней теоретической оценки сложности перебора, которая равна #ccp.

DPccp поддерживает таблицу $BestPlan(S)$ и вместо перебора всех разбиений подмножеств рассматривает ровно csg-cmp-pairs:

1. Инициализация: $BestPlan(\{R_i\}) = R_i$ для всех R_i.
2. Перебор всех csg-cmp-pairs (S_1, S_2); положим $S = S_1 \cup S_2$.
3. Для текущей пары берутся $p_1 = BestPlan(S_1)$, $p_2 = BestPlan(S_2)$, строится кандидат $CurrPlan = CreateJoinTree(p_1, p_2)$ и обновляется $BestPlan(S)$, если кандидат дешевле.
4. Так как процедура перечисления генерирует только одну ориентацию пары, алгоритм дополнительно учитывает коммутативность соедине-

ния, пробуя $CreateJoinTree(p_2, p_1)$.

Algorithm 3 DPccp

```
Input: a connected query graph with relations R = {R_0, ..., R_{n-1}}
Output: an optimal bushy join tree
for all R_i ∈ R do
    BestPlan({R_i}) ← R_i
end for
for all csg-cmp-pairs (S_1, S_2), S = S_1 ∪ S_2 do
    InnerCounter ← InnerCounter + 1
    p_1 ← BestPlan(S_1)
    p_2 ← BestPlan(S_2)
    CurrPlan ← CreateJoinTree(p_1, p_2)
    if cost(BestPlan(S)) > cost(CurrPlan) then
        BestPlan(S) ← CurrPlan
    end if
    CurrPlan ← CreateJoinTree(p_2, p_1)
    if cost(BestPlan(S)) > cost(CurrPlan) then
        BestPlan(S) ← CurrPlan
    end if
end for
return BestPlan({R_0, ..., R_{n-1}})
```

Преимущества:

1. Достижение нижней границы перебора: DPccp рассматривает ровно #ccp; в тексте подчёркивается, что это является нижней границей для

DP.

Недостатки:

1. Худший случай всё равно экспоненциален: на полных графах число #ccp очень быстро возрастает, и DPccp также становится очень дорогим.
2. Сложность реализации: нужно эффективно перечислять csg-cmp-pairs без дубликатов и в порядке, корректном для DP, чтобы перед (S_1, S_2) уже были рассмотрены все непустые подмножества.

Сложность DPccp от числа отношений n:

$$I_{\text{DPccp}}^{\text{chain}}(n) = n^3$$

$$I_{\text{DPccp}}^{\text{cycle}}(n) = n^3$$

$$I_{\text{DPccp}}^{\text{star}}(n) = n2^n$$

$$I_{\text{DPccp}}^{\text{clique}}(n) = 3^n$$

В статье алгоритм DPccp перечисляет csg в порядке, согласованном с DP, используя нумерацию, построенную поиском в ширину, вершин и запрет на включение вершин с меткой меньше стартовой, чтобы не порождать дубликаты. Для каждого S_1 процедура `EnumerateCmp` строит вторую часть пары S_2 только внутри дополнения $V \setminus S_1$: стартует с одиночных вершин из окрестности (S_1) и затем рекурсивно расширяет их, добавляя подмножества

соседей уже построенного S_2 (то есть всегда растит связный компонент). Параметр исключения X выбирается так, чтобы S_2 не содержал вершин с меткой меньше любой вершины из S_1 (условие порядка $min(S_1) < min(S_2)$, где min(S) - минимальный номер вершины, входящей в S), что устраняет симметричные дубликаты (S_1, S_2) / (S_2, S_1). Из-за старта из окрестности (S_1) и связного расширения гарантируется смежность S_2 к S_1 (есть ребро между компонентами), поэтому перечисляются ровно допустимые csg-cmp-пары.

3.9. DPhyp: динамическое программирование на гиперграфах

В другой статье, от тех же авторов [8] замечается, что DPccp эффективен для связных графов запросов, в котором все соединения inner join и являются бинарными, т.е каждое условие включает ровно две таблицы: Однако реальные запросы содержат *сложные предикаты*, связывающие сразу несколько отношений, и *non-inner joins* с ограничениями на перестановки. DPhyp обобщает DPccp на эти случаи за счёт представления запроса в виде *гиперграфа*.

3.9.1. Гиперграф и csg-cmp-пары

Запрос представим гиперграфом $H = (V, E)$, где V — таблицы, а гиперребро $e = (U, W)$ связывает две непересекающиеся группы $U, W \subseteq V$, $U \cap W = \varnothing$ (частный случай $|U| = |W| = 1$ даёт обычное ребро).

Комплементарная пара определяется аналогично, только вместо рёбер

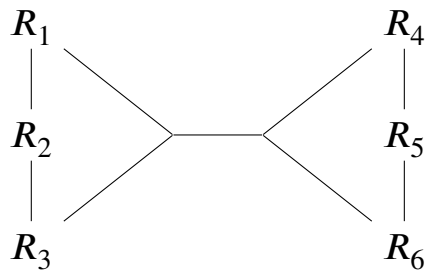


Рис. 3.1: Пример гиперграфа

гиперёбра.

3.9.2. Идея перечисления

DPhyp перечисляет аналогично только ccp. Вводится линейный порядок на отношениях $\prec$ и правило уникальности: рассматриваются только пары с $\min(S_1) \prec \min(S_2)$. Порядок нужен, чтобы симметричные дубликаты не порождались.

Компоненты строятся рекурсивным расширением через *окрестность* $N(S, X)$ — разрешённые узлы, достижимые из S по гиперрёбрам, исключая запрещённые X.

Пусть есть цепочка $R_1 - R_2 - R_3$, цепочка $R_4 - R_5 - R_6$ и гиперребро между $\{R_1, R_2, R_3\}$ и $\{R_4, R_5, R_6\}$. Тогда $(S_1, S_2) = (\{R_1, R_2, R_3\}, \{R_4, R_5, R_6\})$ — валидная csg-cmp-pair: обе стороны связны и соединяемы гиперребром. Характерная ситуация: при $S_1 = \{R_1, R_2, R_3\}$ окрестность даёт вход $\min(\{R_4, R_5, R_6\}) = R_4$, и процедура `EnumerateCmpRec` достраивает S_2 до полной тройки, после чего пара становится соединяемой.

3.9.3. DPhyp

Algorithm 4 DPhyp (перечисление csg-cmp-пар)

```
1:  Input: Гиперграф запроса H = (V, E), порядок ≺ на V
2:  Output: an optimal bushy join tree.
3:  global dp[·]    ▷ DP-таблица: множество отношений → оптимальный
    план
4:  function Solve
5:      for all v ∈ V do
6:          dp[{v}] ← Scan(v)
7:      end for
8:      for all v ∈ V in descending ≺ do
9:          EmitCsg({v})
10:         B_v ← { w ∈ V | w ≺ v } ∪ {v}
11:         EnumerateCsgRec({v}, B_v)
12:     end for
13:     return dp[V]
14: end function
15: function EnumerateCsgRec(S_1, X)
16:     Y ← Neighborhood(S_1, X)
17:     for all N ⊆ Y with N ≠ ∅ do
18:         if Connected(S_1 ∪ N) then
19:             EmitCsg(S_1 ∪ N)
20:             EnumerateCsgRec(S_1 ∪ N, X ∪ Y)
21:         end if
22:     end for
23: end function
24: function EmitCsg(S_1)
```

Для не inner join порядок перестановок ограничен семантикой, поэтому часть разбиений заведомо не корректна.

Идея статьи: закодировать такие ограничения в структуре гиперграфа,добавляя ограничивающие гиперребра, чтобы алгоритм DP перечислял только допустимые комбинации.

3.10. Генетический подход

В работе [9] предлагается генетический алгоритм для оптимизации запросов соединения как альтернатива классическому динамическому программированию типа DPsize. Качество построенного дерева плана определяется стоимостной моделью, которая переводится в функцию приспособленности. Алгоритм пытается её минимизировать. Алгоритм поддерживает популяцию планов кандидатов, итеративно применяя селекцию, перемещение и мутацию.

Ключевые идеи:

1. Два пространства поиска:

   (a) L — только левосторонние планы (линейные деревья).

   (b) A — более общее пространство, включающее ветвистые планы.

   Оптимальный план часто лежит вне L, поэтому для выигрыша по качеству требуется искать в A.

2. Кодирование планов:

   (a) Для L: хромосома — упорядоченный список генов вида (отношение, метод соединения).

(b) Для A: хромосома — упорядоченный список генов, где каждый ген соответствует конкретному соединению из графа запроса, вместе с методом соединения и информацией о порядке.

3. Локальная селекция: вместо глобальной селекции используется схема локальной окрестности, где отбор партнёров для скрещивания происходит внутри локального окружения хромосомы.

4. Операторы поиска:

   (a) Мутации: (i) случайная смена метода соединения; (ii) локальная перестановка соседних генов.

   (b) Перемещение: M2S (две перестановки) и CHUNK (перенос случайного непрерывного фрагмента генов).

Преимущества

1. Масштабируемость для больших запросов: алгоритм не требует DP-таблиц экспоненциального размера и остаётся применимым там, где DPsize становится непрактичным из-за потребления памяти.

2. Возможность улучшать планы относительно левосторонних деревьев: поиск в A позволяет находить ветвистые планы, которые по стоимости могут быть лучше лучшего левостороннего плана.

3. Хорошая параллелизуемость: популяционная природа генетического подхода естественно переносится на параллельную архитектуру с небольшими коммуникационными затратами.

Недостатки

1. Нет гарантий оптимальности и стабильности: по мере роста размера запроса качество решений и устойчивость сходимости ухудшаются из-за резкого роста пространства стратегий.
2. Чувствительность к параметрам: качество зависит от размера популяции, выбора операторов перемещения/мутации и схемы селекции; неудачные настройки дают деградацию.
3. Затраты на оценку приспособленности: нужно многократно вычислять стоимость планов для большого числа хромосом, что может доминировать во времени оптимизации.

3.11. Использование графовых топологий при оценке эвристик

В пособии [10] автор использует типовые топологии (цепь, цикл, звезда, полный граф) как контролируемые модели различных классов графов соединений, чтобы сравнить поведение алгоритмов на структурах с разной плотностью рёбер и разными размерами. Отдельно подчёркивается, что эффективность оптимизаторов зависит от структуры графа. Например, цепь и звезда с одинаковым числом таблиц имеют разное время планирования.

Подход: жадная эвристика и сравнение по топологиям

Основной исследуемый алгоритм это жадный подход, описанный выше.

Ключевые идеи оценки

1. Сравнить жадный подход и динамическое программирование по двум метрикам: (i) стоимость найденного плана, (ii) время оптимизации (runtime).

2. Выполнить сравнение на пяти графах: цепь, цикл, звезда, полный граф (синтетические топологии) и бенчмарке IMDB (реальный граф отношений), чтобы выявить влияние топологии на качество/время.
3. Выявлено эмпирическое правило: для простых топологий (цепь и цикл) динамическое программирование доминирует по стоимости и часто по времени планирования, а для более сложных (звезда, полный граф) динамическое программирование остаётся лучшим по стоимости, но начинает резко проигрывать по времени планирования при росте числа отношений.

Преимущества жадного подхода

1. Полиномиальное время планирования и хорошая масштабируемость по времени планирования на больших запросах.
2. Топологии показывают, что порог по числу таблиц недостаточен — важна форма графа соединений.

Недостатки

1. Качество планов по стоимости хуже, чем у DP почти на всех топологиях.
2. На IMDB жадный подход нестабилен по стоимости. Автор фиксирует большую разницу стоимости относительно синтетических топологий и существенно более высокие стоимости у GOO на большинстве размеров.

3.12. Адаптивная оптимизация очень больших запросов соединения

В статье [11] предлагается адаптивный фреймворк оптимизации порядка соединений, который выбирает (и переключает) стратегию планирования по сложности графа соединений и заданному бюджету перебора, чтобы для типичных запросов получать оптимум, а для больших деградировать по качеству плавно и предсказуемо по времени оптимизации.

Под адаптивностью в работе понимается не фиксированное правило вида DP до n таблиц, дальше эвристика, а выбор алгоритма по структуре (топологии) графа запроса и контроль затрат оптимизации через бюджет перечисления. В результате малые/простые запросы решаются точно, а большие обрабатывается эвристически, но с контролируемым временем оптимизации.

Ключевые идеи и особенности адаптивного планирования

1. Оценка сложности через число связных подграфов. Авторы считают #ccp графа соединений. Это число совпадает с размером полной DP-таблицы (коррелирует со временем оптимизации). Если число связных подграфов не превышает заданного порога, то графовый DP (DPhyp) считается быстрым, и запрос оптимизируется точно.

2. Линеаризация для средних запросов. Когда полный DP становится слишком дорогим, вводится линеаризация пространства поиска: сначала строится линейный порядок отношений, затем DP ограничивается только связными подцепочками этого порядка, вместо произвольных подмножеств. Это уменьшает размер DP-таблицы с $O(2^n)$

до $O(n^2)$ (и делает время порядка $O(n^3)$). Качество плана зависит от выбранного порядка: при “плохой” линеаризации некоторые хорошие порядки соединений становятся недостижимыми. Кроме того, гипер-рёбра не выражаются линейно, поэтому linearized DP применима только к обычным графам соединений.

3. GOO + локальная оптимизация больших поддеревьев для очень больших запросов. Для очень больших запросов строится начальный план жадной эвристикой (GOO), после чего выполняется итеративное улучшение: выбранные (дорогие) поддеревья перепланируются более точным методом DP, но только до размера k.

Преимущества

1. Оптимальность для простого случая: если граф достаточно простой по #ccp бюджету, фреймворк запускает DPHyp и находит оптимальный порядок.
2. Масштабирование до тысяч отношений: при росте размера запроса происходит переход к менее дорогим стадиям.
3. Плавная деградация качества.
4. Учитывается форма графа запроса: выбор алгоритма определяется не только числом начальных отношений, что точнее отражает реальную сложность перечисления.

Недостатки

1. Линеаризация неприменима при внешних соединениях и гиперрёбрах
2. Зависимость качества от линеаризации: выбранный линейный порядок ограничивает пространство планов, некоторые хорошие порядки

соединений становятся недостижимыми.

3. Локальность улучшений в GOO-DP: перепланирование поддеревьев размера k не даёт полной свободы глобальных перестановок между различными поддеревьями, поэтому итог может оставаться далёким от оптимума.
4. Пороговые параметры являются эвристическими: бюджет, границы и k определяют поведение и требуют настройки под конкретную систему и модель стоимости.

3.13. Архитектура планировщика PostgreSQL

Практическая часть данной работы выполнена на базе СУБД PostgreSQL — наиболее распространённой реляционной системы с открытым исходным кодом. Для понимания экспериментальных результатов необходимо описать, как теоретические алгоритмы перечисления планов реализованы в данной системе и какие инженерные ограничения при этом возникают.

Конвейер планирования. Планировщик PostgreSQL реализует алгоритм DPsize поуровнево. Пусть дан запрос с n отношениями. Планирование выполняется в цикле по уровню $s = 2, 3, \ldots, n$, где s — число базовых отношений в подплане. На каждом уровне конвейер состоит из четырёх этапов:

1. **Перечисление пар** — процедура перебирает все разбиения целевого множества S ($|S| = s$) на два непересекающихся подмножества S_1 и S_2, для которых уже найдены подпланы, и которые связаны хотя бы

одним предикатом соединения. Перебор выполняется в фиксированном порядке: сначала левосвязанные разбиения ($|S_1| = s - 1$, $|S_2| = 1$), затем ветвистые ($|S_1| = s - 2$, $|S_2| = 2$), …

2. **Построение путей** — для каждой допустимой пары (S_1, S_2) вызывается процедура, которая конструирует конкретные физические пути исполнения соединения (вложенный цикл, хеш-соединение, соединение слиянием) для обоих направлений: S_1 как внешний и S_2 как внутренний аргумент, и наоборот. Эта процедура является самой дорогой частью планирования: по данным профилирования, на неё приходится свыше 90% времени t_{plan}.

3. **Инкрементальный отбор путей** — каждый построенный путь немедленно сравнивается со всеми ранее сохранёнными путями для того же множества S. Путь отклоняется, если существует сохранённый путь, доминирующий по всем критериям (стоимость, порядок сортировки, параметризация, оценка числа строк). Сравнение стоимостей выполняется с допуском $\varepsilon = 1\%$: два пути считаются эквивалентными по стоимости, если $|\text{Cost}(p_1) - \text{Cost}(p_2)| < \varepsilon \cdot \min(\text{Cost}(p_1), \text{Cost}(p_2))$. При эквивалентности по всем критериям сохраняется путь, поступивший *раньше*. Отклонённые пути освобождаются из памяти и не могут быть восстановлены.

4. **Финализация уровня** — после перебора всех разбиений для каждого вновь созданного множества S выбирается путь с минимальной оценочной стоимостью из числа выживших. Указатель на этот путь фиксируется и используется при построении подпланов на уровне

$s + 1$.

Существенно, что этапы 2 и 3 выполняются *одновременно*: каждый новый путь проходит инкрементальный отбор сразу после построения, а не по завершении перебора всех разбиений. Поэтому пути из более ранних разбиений (левосвязанных) уже присутствуют в множестве сохранённых, когда поступают пути из более поздних разбиений (ветвистых). Следствия этого порядка описаны ниже.

Порядок перебора как скрытый параметр. Пусть для множества S рассмотрены два разбиения: $S = S_1^{(a)} \cup S_2^{(a)}$ (левосвязанное, перебрано первым) и $S = S_1^{(b)} \cup S_2^{(b)}$ (ветвистое, перебрано позже). Обозначим построенные пути p_a и p_b соответственно. Если $\mathrm{Cost}(p_a) \approx \mathrm{Cost}(p_b)$ с точностью ε, то p_a остаётся в множестве сохранённых, а p_b отклоняется. При обратном порядке перебора выжил бы p_b.

При точных оценках кардинальностей этот эффект незначителен: стоимости различных планов, как правило, разделены. Однако при ошибочных оценках (см. предыдущую секцию) стоимости конкурирующих путей часто попадают в ε-окрестность, и порядок перебора фактически предопределяет результат. Поскольку левосвязанные разбиения обрабатываются первыми на каждом уровне, планировщик PostgreSQL имеет систематическое структурное смещение в пользу левосвязанных планов.

Число вызовов процедуры построения путей. Количество пар (S_1, S_2), для которых вызывается процедура построения путей, определяет время планирования t_{plan}. На уровне s число допустимых разбиений зависит от топологии графа соединений G. Для полного графа из n вершин на уровне s рассматриваются $\binom{n}{s} \cdot (2^s - 2)/2$ разбиений, с максимумом при $s \approx n/2$.

Распределение числа вызовов по уровням имеет характерную форму: малое число на крайних уровнях ($s = 2$ и $s = n$) и максимум на средних уровнях, где сосредоточена основная вычислительная нагрузка планирования. Любая стратегия ускорения t_{plan} должна сокращать число вызовов именно на этих средних уровнях.

Переключение на генетический алгоритм. PostgreSQL использует пороговое правило: если число отношений $n \geqslant 12$, планировщик переключается с DPsize на генетический алгоритм GEQO. Это переключение имеет два недостатка. Во-первых, порог задан по числу отношений n, а не по сложности графа: запрос-цепь из 20 таблиц имеет $\#\text{csg} = O(n^2) = 400$ и тривиален для DP, тогда как запрос-клика из 12 таблиц имеет $\#\text{csg} = 2^{12} - 1 = 4095$ и значительно сложнее. Во-вторых, переключение происходит скачкообразно: при $n = 11$ выполняется точный перебор, а при $n = 12$ запускается стохастический алгоритм без гарантий качества, строящий только левосвязанные планы.

Описанные свойства архитектуры — зависимость от порядка перебора, доминирование процедуры построения путей во времени планирования и грубое переключение стратегий — определяют как возможности, так и ограничения для эвристических улучшений планировщика.

3.14. Оценки в модели стоимости

В статье [1] авторы исследуют, насколько хорошо оптимизаторы промышленных СУБД справляются с выбором порядка соединений на реальных данных. Используется Join Order Benchmark (JOB) на базе данных

IMDb: 113 запросов с множественными соединениями и коррелированными фильтрами. Модифицированный PostgreSQL получает доступ к *истинным* кардинальностям промежуточных результатов, что позволяет отделить влияние ошибок оценок от других компонентов оптимизатора.

Сбор и представление статистик. Оценки селективностей и кардинальностей не вычисляются аналитически, а строятся по статистикам, собранным командой `ANALYZE`. При выполнении этой команды СУБД для каждого столбца каждой таблицы производит выборку строк и вычисляет три структуры:

1. **Число различных значений** (n_{distinct}) — оценка мощности множества значений столбца. Используется для оценки селективности условия равенства: $\text{sel}(A = c) \approx 1/n_{\text{distinct}}(A)$ при предположении равномерного распределения.

2. **Список наиболее частых значений** (MCV, Most Common Values) — упорядоченный список пар $(value_i,\ f_i)$, где f_i — доля строк таблицы, содержащих значение $value_i$. Размер списка ограничен параметром `default_statistics_target` (обозначим d; по умолчанию $d = 100$). Для конкретного значения c, попавшего в MCV-список, селективность оценивается точно: $\text{sel}(A = c) = f_c$.

3. **Гистограмма равновысоких корзин** (equi-height histogram) — массив из d граничных значений, разбивающих область значений столбца (за вычетом MCV-значений) на $d - 1$ корзин приблизительно одинаковой частоты. Используется для оценки селективности диапазонных условий: $\text{sel}(A > c)$ оценивается по доле корзин, лежащих правее c, с

линейной интерполяцией внутри граничной корзины.

Параметр d определяет точность статистик: чем он больше, тем длиннее MCV-список и тем мельче корзины гистограммы. Увеличение d уменьшает ошибки оценок для одиночных предикатов на одном столбце, однако *не решает* проблему оценки совместных распределений нескольких столбцов из разных таблиц. Статистики собираются независимо для каждого столбца, и оценка селективности соединения $\mathrm{sel}(R_i.A = R_j.B)$ строится в предположении независимости:

$$\mathrm{sel}(R_i.A = R_j.B) \;\approx\; \frac{1}{\max\bigl(n_{\mathrm{distinct}}(A),\; n_{\mathrm{distinct}}(B)\bigr)}.$$

При наличии корреляций между атрибутами разных таблиц (например, фильмы определённого жанра чаще снимаются определёнными актёрами) это предположение нарушается, и ошибки каскадируются мультипликативно по цепочке соединений.

Ошибки оценок кардинальностей. Стоимостная модель оптимизатора вычисляет $\mathrm{Cost}(T)$ по оценкам кардинальностей промежуточных результатов $|\hat{S}|$ для каждого подплана S в дереве T. Авторы показывают, что для запросов с k соединениями ошибки кардинальности растут мультипликативно: если оценка каждого единичного предиката ошибается в q раз, то после k соединений накопленная ошибка может достигать q^k. На JOB для запросов с 5–17 таблицами зафиксированы ошибки в 10^2–10^6 раз относительно истинных кардинальностей промежуточных результатов. При подстановке истинных кардинальностей планы оптимизатора улучшаются на порядки по времени исполнения, тогда как изменение параметров самой

стоимостной функции (соотношения весов I/O и CPU) влияет значительно слабее.

Роль стоимости как ориентира. Стоимость $\mathrm{Cost}(T)$ задумана как величина, монотонно коррелирующая с фактическим временем исполнения t_{exec}. Однако при ошибочных оценках кардинальностей эта корреляция нарушается: два плана T_1 и T_2 могут иметь близкие оценочные стоимости $\mathrm{Cost}(T_1) \approx \mathrm{Cost}(T_2)$, но кратно отличающееся фактическое время исполнения.

Пример. Рассмотрим запрос с четырьмя отношениями R_1, R_2, R_3, R_4 и два плана:

- $T_1 = (R_1 \bowtie R_2 \bowtie R_3) \bowtie R_4$ (левосвязанный),
- $T_2 = (R_1 \bowtie R_2) \bowtie (R_3 \bowtie R_4)$ (ветвистый).

Пусть оптимизатор оценивает $|R_3 \bowtie R_4| = 500$, тогда как фактически $|R_3 \bowtie R_4| = 50\,000$ из-за корреляции между атрибутами, не отражённой в статистике. Стоимости обоих планов оказываются близки: $\mathrm{Cost}(T_1) \approx \mathrm{Cost}(T_2)$. Однако при исполнении T_2 промежуточный результат $R_3 \bowtie R_4$ оказывается в 100 раз больше ожидаемого, что приводит к $t_{\mathrm{exec}}(T_2) \approx 3 \cdot t_{\mathrm{exec}}(T_1)$.

Эмпирическое правило для сравнения подпланов. Из наблюдения о мультипликативном росте ошибок следует практический вывод: для подпланов малого размера ($|S| \leqslant 2$ базовых отношений) ошибки оценок ещё невелики, и сравнение по Cost остаётся надёжным. Для подпланов большего размера ошибки кардинальностей каскадируются, и сравнение по оценке кардинальности $|\hat{S}|$ оказывается более робастным ориентиром, чем сравнение по $\mathrm{Cost}(S)$, поскольку кардинальность ошибается монотоннее,

без резких инверсий, характерных для стоимостной модели при больших ошибках.

Зависимость результата от порядка перебора. Алгоритмы динамического программирования (DPsize, DPhyp) строят таблицу $BestPlan(S)$, последовательно перебирая разбиения множества S на подмножества. Для каждого нового кандидата выполняется сравнение с уже найденным планом для того же S. В реализациях промышленных СУБД это сравнение не точное, а приближённое: два плана считаются эквивалентными, если их оценочные стоимости различаются менее чем на фиксированную долю ε (в PostgreSQL $\varepsilon = 1\%$). При совпадении побеждает тот план, который был найден *раньше*.

Это означает, что результат планирования зависит не только от множества рассмотренных планов, но и от *порядка*, в котором они генерируются. В DPsize на каждом уровне s сначала перебираются левосвязанные разбиения $(S_{s-1},\ S_1)$, затем ветвистые $(S_{s-2},\ S_2)$, … Левосвязанные кандидаты попадают в $BestPlan$ первыми и получают структурное преимущество при сравнении с точностью ε. При наличии ошибок оценок кардинальностей стоимости конкурирующих планов часто оказываются в ε-окрестности друг друга, и порядок перебора фактически *предопределяет* выбор, независимо от истинного качества планов. Эта зависимость является свойством реализации, а не алгоритма: при идеальных оценках кардинальностей она не проявляется, поскольку стоимости различных планов разделены.

Выводы из [1]:

1. Главный источник плохих планов — ошибки оценок кардинальностей, а не неточность самой стоимостной функции.

2. При доступе к истинным кардинальностям оптимизатор почти всегда находит план, близкий к оптимальному.
3. Полный перебор (DP) даёт преимущество над ограниченными стратегиями поиска даже при неточных оценках: он реже пропускает хорошие планы.
4. Катастрофические деградации возникают при сочетании больших ошибок кардинальностей с рискованными выборами (глубокие деревья вложенных циклов), и их частота растёт с числом соединений в запросе.

3.15. Открытые вопросы

Описанные подходы оставляют открытым следующий вопрос. Алгоритмы точного перечисления (DPsize, DPccp, DPhyp) гарантируют оптимальность найденного плана, но время планирования растёт экспоненциально с числом таблиц. Эвристики (GOO, GEQO) масштабируются полиномиально, но не гарантируют качества плана. Адаптивный фреймворк [11] предлагает переключение между стратегиями, но использует линеаризацию, неприменимую к гиперграфам и внешним соединениям. Переключение по числу таблиц n в PostgreSQL ($n \geqslant 12 \rightarrow$ GEQO) игнорирует топологию графа соединений.

Ни один из описанных подходов не использует *структуру графа соединений* для безопасного сокращения пространства поиска перед точным перебором. Данная работа предлагает адаптивный подход, в котором подграфы с предсказуемыми свойствами (малая кардинальность, высокий контраст

с соседями) контрактируются до основного DP, уменьшая число вершин графа и, как следствие, число вызовов самой дорогой процедуры планировщика.

4. Основная часть

4.1. Окружение

Все эксперименты выполнены на одной машине с фиксированной конфигурацией, чтобы обеспечить воспроизводимость и сопоставимость результатов.

Аппаратное обеспечение.

- Процессор: Intel Core Ultra 7 258V (8 ядер).
- Оперативная память: 32 ГБ.
- Операционная система: Linux Ubuntu 25.04.

СУБД. PostgreSQL версии 18.2 (текущая на момент экспериментов ветка разработки).

Бенчмарк. Используется Join Order Benchmark (JOB) [1] — набор из 113 запросов к базе данных IMDb (Internet Movie Database), содержащей информацию о фильмах, актёрах, ключевых словах и связанных сущностях. Запросы JOB содержат от 5 до 17 соединений, коррелированные фильтры и разнообразные топологии графов соединений (цепи, звёзды, циклы и их комбинации), что делает бенчмарк стандартным инструментом для оценки качества планировщиков соединений.

Подготовка данных. Перед каждым запуском экспериментов выполняются два шага:

1. **Сбор статистик.** Команда `ANALYZE VERBOSE` обновляет статистики всех таблиц. Параметр `default_statistics_target` установлен в 10 000 (по умолчанию — 100). Это значение определяет максимальное число элементов в гистограммах распределений и списках наиболее частых значений, используемых оптимизатором для оценки селективностей. Увеличение до 10 000 минимизирует ошибки оценок одиночных предикатов, хотя не устраняет ошибки, вызванные корреляциями между атрибутами разных таблиц при многотабличных соединениях.

2. **Прогрев кэша.** Все таблицы, индексы, материализованные представления и TOAST-таблицы загружаются в буферный кэш СУБД вызовом `pg_prewarm(oid, 'buffer')`. Это исключает влияние дискового ввода-вывода на измерения: все обращения к данным во время экспериментов обслуживаются из оперативной памяти.

Конфигурация СУБД. Параметры PostgreSQL выбраны так, чтобы устранить влияние побочных эффектов (дисковый ввод-вывод, нехватка памяти для операторов) на время исполнения запросов, изолируя эффект от выбора порядка соединений:

- `shared_buffers = 12 GB` — размер буферного кэша СУБД. Достаточен для размещения всей базы IMDb в оперативной памяти, что вместе с прогревом гарантирует отсутствие дисковых чтений.

- `work_mem = 8 GB` — объём памяти, выделяемый каждой операции сор-

тировки или хеширования. При таком значении хеш-соединения выполняются в один проход (без записи промежуточных данных на диск), а сортировки не используют внешнюю сортировку. Это устраняет ступенчатые нелинейности в стоимостной модели, возникающие при переходе между однопроходным и многопроходным режимами.

- `effective_cache_size = 8GB` — подсказка оптимизатору об объёме доступного кэша операционной системы. Влияет на оценку стоимости индексных сканирований: при большом значении оптимизатор предполагает, что данные индексов с высокой вероятностью находятся в кэше.
- `max_parallel_workers = 8`, `max_parallel_workers_per_gather = 8`, `max_worker_processes = 8`, `max_parallel_maintenance_workers = 8` — параметры параллелизма, разрешающие использование всех доступных ядер процессора для параллельного исполнения операторов плана.

Метрики. Каждый запрос характеризуется тремя временными величинами:

- t_{plan} — время планирования (выбор порядка соединений и построение плана),
- t_{exec} — время исполнения построенного плана на данных,
- $t_{\text{total}} = t_{\text{plan}} + t_{\text{exec}}$ — суммарное время обработки запроса.

Все времена измеряются средствами PostgreSQL (`EXPLAIN (ANALYZE, TIMING)`) и приводятся в миллисекундах. Результаты каждого алгоритма нормализуются относительно стандартного планировщика PostgreSQL

(DPsize): значение < 1 означает улучшение, > 1 — деградацию.

Критерий сложности графа соединений. Время работы алгоритмов перечисления подпланов определяется не столько числом n, сколько числом связных подграфов $\#\mathrm{csg}(G)$ графа G [11]. Для цепи из n вершин $\#\mathrm{csg} = O(n^2)$, тогда как для полного графа $\#\mathrm{csg} = 2^n - 1$. Поэтому вместо порогового правила по n адаптивный алгоритм использует $\#\mathrm{csg}(G)$ для выбора стратегии планирования, что позволяет учитывать топологическую структуру запроса.

4.2. Подход 1. Жадные алгоритмы с различными критериями сравнения

Мотивация. Жадный алгоритм GOO (описан в секции «Предшествующие работы») на каждом шаге выбирает пару отношений с минимальным значением некоторого критерия и соединяет их. Классическая реализация использует оценочную стоимость Cost в качестве критерия. Однако, как показано в секции «Оценки в модели стоимости», при увеличении числа соединений ошибки оценок кардинальностей каскадируются и стоимость Cost теряет способность различать планы по истинному качеству. Возникает вопрос: можно ли подобрать критерий сравнения, более робастный к ошибкам оценок?

Реализованы и протестированы три варианта GOO, различающиеся только критерием выбора лучшей пары на каждом шаге. Во всех трёх вариантах структура алгоритма одинакова: на каждой итерации перебираются все пары отношений, связанных предикатом соединения, для каждой пары

строится соединение (вызов процедуры построения путей), и пара с наилучшим значением критерия объединяется. Сложность каждого варианта — $O(n^3)$ вызовов процедуры построения путей, где n — число начальных отношений.

Вариант 1: GOO по стоимости ($\mathrm{GOO}_{\mathrm{cost}}$). Критерий — оценочная стоимость результата соединения:

$$(R_i^*,\ R_j^*)\ =\ \arg\min_{(R_i,R_j)} \mathrm{Cost}(R_i \bowtie R_j).$$

Это классическая реализация GOO, в которой на каждом шаге выбирается пара с минимальной стоимостью по модели СУБД.

Вариант 2: GOO по кардинальности ($\mathrm{GOO}_{\mathrm{card}}$). Критерий — оценка объёма результата соединения:

$$(R_i^*,\ R_j^*)\ =\ \arg\min_{(R_i,R_j)} \left(|\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j)\right),$$

где $|\widehat{R_i \bowtie R_j}|$ — оценочная кардинальность, а $w(R_i \bowtie R_j)$ — средняя ширина строки результата (в байтах). Произведение $|\widehat{R_i \bowtie R_j}| \cdot w$ оценивает объём данных промежуточного результата и является более грубым, но более робастным сигналом, чем Cost: кардинальность ошибается монотоннее, без резких инверсий, характерных для стоимостной модели при больших ошибках оценок.

Вариант 3: GOO с комбинированным критерием ($\mathrm{GOO}_{\mathrm{comb}}$). Критерий адаптивно переключается в зависимости от размера аргументов соединения. Пусть $|S|$ — число базовых отношений в подплане S. Пара (R_i, R_j)

называется *простой*, если $|R_i| \leqslant 2$ и $|R_j| \leqslant 2$. Критерий выбора:

$$(R_i^*,\ R_j^*) \ = \ \arg\min_{(R_i,R_j)} \begin{cases} \mathrm{Cost}(R_i \bowtie R_j), & \text{если пара простая,} \\ |\widehat{R_i \bowtie R_j}| \cdot w(R_i \bowtie R_j), & \text{иначе.} \end{cases}$$

Логика переключения основана на наблюдении из [1]: для подпланов малого размера ($|S| \leqslant 2$) ошибки оценок кардинальностей ещё не успели накопиться и стоимостная модель даёт надёжное ранжирование. Для подпланов большего размера ошибки каскадируются, и сравнение по объёму промежуточного результата оказывается более устойчивым. Порог $|S| \leqslant 2$ соответствует подпланам, содержащим не более одного соединения, где мультипликативное накопление ошибок ещё не проявляется.

Результаты. Все три варианта протестированы на 113 запросах JOB. Суммарные отношения к DPsize:

	t_{plan}	t_{exec}	t_{total}	побед из 113
GOO_{cost}	0,85	1,44	1,22	55
GOO_{card}	0,86	1,62	1,34	64
GOO_{comb}	0,81	1,19	1,05	80

Все три варианта сокращают t_{plan} на 14–19% (алгоритм $O(n^3)$ вместо $O(3^n)$ у DPsize), но проигрывают по t_{exec}. Ключевое наблюдение: GOO_{comb} выигрывает у 80 из 113 запросов (71%), тогда как GOO_{cost} — лишь у 55 (49%), а GOO_{card} — у 64 (57%).

Случаи, когда жадные подходы превосходят DP. Несмотря на то что DPsize выполняет полный перебор подмножеств, он оптимизирует оценочную стоимость $\mathrm{Cost}(T)$, а не истинное время исполнения. При значительных

ошибках оценки кардинальностей DP может выбрать план, оптимальный по ошибочным оценкам, но неудачный по фактическому времени. Жадный алгоритм перебирает меньше вариантов, но его выбор на каждом шаге менее подвержен каскадному накоплению ошибок, поскольку опирается на локальное сравнение пар, а не на глобальную стоимость всего дерева.

Пример: запрос 6d ($t_{\text{total}}^{\text{DP}} = 2272$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 212$ мс, улучшение в 10,7 раза). Запрос содержит фильтр по таблице n (name): `name LIKE '%Downey%Robert%'`, оставляющий 2 строки.

DP строит план $k \to mk \to t \to ci \to n$, в котором таблица n стоит последней. Стоимостная модель недооценивает селективность шаблонного предиката `LIKE` и оценивает промежуточный результат ci как небольшой. В действительности до соединения с n кардинальность достигает 785 477 строк, и индексное сканирование n по первичному ключу с проверкой шаблона выполняется 785 477 раз.

GOO_{comb} на первом шаге соединяет n (2 строки после фильтра) с ci через индексное сканирование (243 строки), поскольку эта пара имеет минимальную оценку кардинальности среди всех пар. Далее результат соединяется хеш-соединением с веткой $k \to mk \to t$ (14 165 строк). Промежуточный результат после первого шага — 243 строки вместо 785 477. Время исполнения: 117 мс вместо 2163 мс.

Данный пример иллюстрирует общий механизм: когда одна из таблиц имеет высокоселективный фильтр, который стоимостная модель не в состоянии точно оценить, жадный выбор по кардинальности помещает её в начало цепочки соединений, тогда как DP, доверяя ошибочной модели стоимости, откладывает её на конец.

Случаи деградации жадных подходов. Жадный алгоритм принимает на каждом шаге необратимое решение на основании локальной информации. Если выбранное соединение создаёт промежуточный результат, не позволяющий эффективно фильтровать на последующих шагах, деградация неизбежна.

Пример: запрос 17c ($t_{\text{total}}^{\text{DP}} = 3086$ мс, $t_{\text{total}}^{\text{GOO}_{\text{comb}}} = 10\,112$ мс, деградация в 3,3 раза). На ранних шагах GOO_{comb} соединяет $k \to mk \to t \to mc \to cn$: каждая пара имеет малую оценку кардинальности, и на каждом шаге выбор выглядит оптимальным. Однако построенная ветвь не содержит таблицу *n* (name), имеющую высокоселективный фильтр `name LIKE '%B%'`. Когда на последних шагах присоединяются *ci* (52,5 строки на итерацию, 148 552 итераций) и затем *n* (7 796 926 итераций), фильтрация по *n* применяется поздно, и промежуточные результаты оказываются на порядок больше, чем в плане DP.

DP находит порядок, в котором *ci* соединяется с *n* рано (ещё до *mc* и *cn*), что позволяет фильтру по имени сократить промежуточный результат до 250 строк. Жадный алгоритм не способен предвидеть этот эффект, поскольку на момент принятия решения таблица *n* не связана предикатом с текущим промежуточным результатом.

Почему комбинированный критерий лучше. Сравнение GOO_{cost} и GOO_{card} показывает, что ни один из критериев не доминирует на всех запросах. На запросе 13d GOO_{cost} выигрывает (0,78), а GOO_{card} деградирует (2,04). На запросе 20b — наоборот: GOO_{cost} деградирует (2,74), а GOO_{card} выигрывает (0,74).

Причина в том, что два критерия надёжны в разных диапазонах. *Стои-*

мость Cost учитывает выбор физического алгоритма соединения (индексное сканирование, хеш-соединение) и является точным ориентиром при малых подпланах, где оценки кардинальностей ещё не искажены. *Кардинальность* $|\widehat{S}| \cdot w$ не различает физические алгоритмы, но ошибается монотоннее при росте подплана: она не создаёт резких инверсий (ситуаций, когда план A дешевле B по оценке, но дороже по факту), характерных для стоимостной модели при больших ошибках.

Комбинированный критерий $\mathrm{GOO}_{\mathrm{comb}}$ использует сильные стороны обоих: стоимость — пока ошибки малы ($|S| \leqslant 2$), кардинальность — когда ошибки нарастают ($|S| > 2$). Это подтверждается результатами: $\mathrm{GOO}_{\mathrm{comb}}$ выигрывает на 80 запросах (против 55 и 64), а суммарная деградация t_{exec} снижается с 1,44 и 1,62 до 1,19.

4.3. Подход 2. Итеративная декомпозиция графа соединений на топологии

Мотивация. Как отмечается в [6,11], время работы алгоритмов перечисления подпланов существенно зависит от топологии графа соединений. Для цепи из n вершин алгоритм DPccp перебирает $O(n^3)$ пар связных подграфов, для звезды — $O(n \cdot 2^n)$, для полного графа — $O(3^n)$. Возникает естественная идея: если граф запроса можно разложить на подграфы с известной топологией, то каждый подграф можно спланировать специализированным алгоритмом с меньшей сложностью, чем общий DP для всего графа.

Описание подхода. Входом является граф соединений $G = (\mathcal{R}, \mathcal{E})$. Подход выполняется итеративно: на каждой итерации граф разбивается

на топологические фрагменты, каждый фрагмент планируется, результаты становятся виртуальными вершинами нового графа, и процесс повторяется до тех пор, пока граф не сведётся к одной вершине.

Шаг 1. Разделение на компоненты связности. Граф G разбивается на компоненты связности обходом в ширину (BFS). Каждая компонента обрабатывается независимо.

Шаг 2. Распознавание топологий. Внутри каждой компоненты последовательно выделяются четыре типа подграфов в фиксированном порядке приоритетов:

1. **Плотные подграфы.** Для всех вершин компоненты строится матрица селективностей. Ребро считается «хорошим», если его селективность не превышает порогового значения $\sigma_{\max}$. Из подграфа хороших рёбер итеративно удаляется вершина с минимальной степенью, пока плотность (отношение числа рёбер к максимально возможному) не превысит порог $\delta_{\min}$. Найденные плотные ядра маркируются как отдельные топологии.

2. **Звёзды.** Среди оставшихся вершин ищутся центры звёзд: вершина v считается центром, если её степень $\geqslant 3$ среди немаркированных соседей и $|v| < \overline{|N(v)|}$ (кардинальность v меньше средней кардинальности соседей). Для каждого центра строятся лучи — цепочки соседей, продлеваемые по наименьшей селективности до фиксированной длины.

3. **Циклы.** Среди оставшихся вершин выполняется поиск в глубину (DFS) с фильтрацией рёбер по селективности. Найденные простые

циклы длины $\geqslant 3$ выделяются как отдельные топологии.

4. **Цепи и остаток.** Все оставшиеся немаркированные вершины группируются в компоненты связности. Если подграф компоненты является путевым графом (каждая вершина имеет степень $\leqslant 2$, число рёбер равно числу вершин минус один), он маркируется как цепь; иначе — как подграф общего вида.

Каждая вершина принадлежит ровно одной топологии; приоритет гарантирует, что вершина, захваченная на более раннем этапе, не рассматривается на последующих.

Шаг 3. Планирование каждой топологии. Для каждого выделенного фрагмента вызывается специализированный алгоритм:

- **Цепь:** интервальное DP за $O(n^3)$ — перебираются все подцепочки $[i, j]$ с разбиением по промежуточной точке k.
- **Цикл:** удаляется ребро с максимальной оценочной кардинальностью результата соединения, цикл превращается в цепь и планируется интервальным DP; удалённая вершина присоединяется в конце.
- **Звезда:** лучи планируются как независимые цепи, затем результаты жадно присоединяются к центру по минимальной стоимости.
- **Плотный подграф и общий вид:** DP по подмножествам (DPsub).

Если планирование фрагмента не завершается полным планом (например, из-за исчерпания бюджета), возвращаются частичные подпланы, покрывающие максимальное число вершин фрагмента.

Шаг 4. Перестроение графа. Результат планирования каждой топологии — одно или несколько отношений. Из них строится новый граф

соединений: виртуальные вершины заменяют исходные фрагменты, рёбра определяются наличием предикатов соединения между отношениями из разных фрагментов. Если новый граф содержит более одной вершины, управление возвращается к шагу 2.

Шаг 5. Финальная сборка. После схождения итеративного процесса компоненты связности объединяются жадным алгоритмом GOO с комбинированной метрикой сравнения (по стоимости для малых подпланов, по кардинальности для больших).

Результаты. Подход протестирован на всех 113 запросах JOB. Суммарные отношения к стандартному DPsize:

- t_{plan}: 0,83 (экономия 17%),
- t_{exec}: 3,55 (деградация в 3,6 раза),
- t_{total}: 2,55 (деградация в 2,5 раза).

Из 113 запросов лишь 32 показали улучшение t_{total} относительно DP; 81 запрос деградировал, причём 10 запросов — более чем в 5 раз. Экономия t_{plan} достигается за счёт меньшего числа вызовов процедуры построения путей в каждом фрагменте, однако деградация t_{exec} её полностью перекрывает.

Анализ причин деградации. Деградация t_{exec} вызвана тремя структурными проблемами подхода.

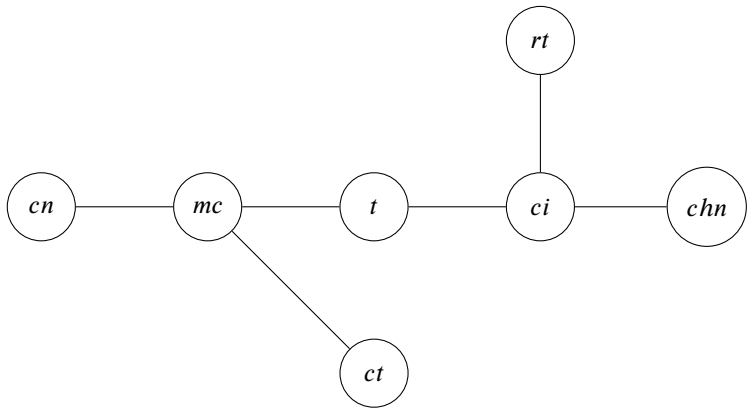
1. Потеря параметризованных путей через hub-вершины. В нормализованных реляционных схемах существуют центральные (hub) таблицы, к которым присоединяются многие другие через внешние ключи. При декомпозиции hub-вершина попадает в один фрагмент, а соседние таблицы из других фрагментов теряют возможность использовать индексное скани-

рование по первичному ключу hub-таблицы. Вместо параметризованного индексного сканирования (стоимость $O(\log N)$ на строку) оптимизатор вынужден использовать хеш-соединение или вложенный цикл без индекса (стоимость $O(N)$).

2. Жёсткий приоритет распознавания. Порядок распознавания (плотные → звёзды → циклы → цепи) является жадным решением без возврата. Если подграф на более раннем этапе захватил вершину, которая критически важна для остальной части графа, качество итогового плана ухудшается.

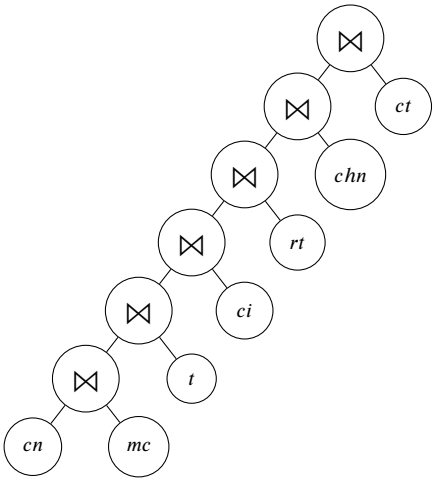
3. Каскадирование ошибок между итерациями. Виртуальная вершина, созданная на итерации k, несёт оценку кардинальности, полученную при планировании фрагмента. Если эта оценка ошибочна, она искажает все решения на итерации $k + 1$. В стандартном DPsize такого каскада нет: все разбиения рассматриваются одновременно на каждом уровне.

Пример деградации: запрос 10a. Запрос 10a соединяет 7 таблиц: cn (company_name), mc (movie_companies), t (title), ci (cast_info), rt (role_type), chn (char_name), ct (company_type). Граф соединений:



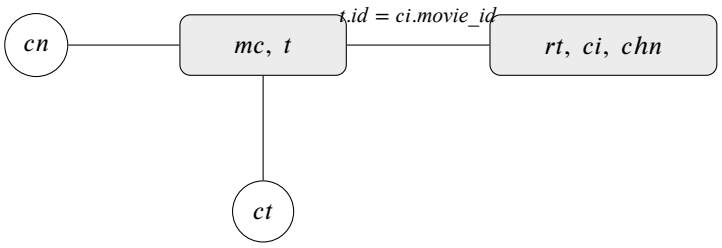
Кардинальности: $|ci| \approx 36 \cdot 10^6$, $|t| \approx 2{,}5 \cdot 10^6$, $|mc| \approx 2{,}6 \cdot 10^6$. Таблицы rt и ct — справочники (единицы строк после фильтрации), cn и chn — справочники с фильтрами по имени.

План DPsize (t_{plan} = 93 мс, t_{exec} = 176 мс). Стандартный планировщик строит левосвязанную цепочку, начиная с отфильтрованной таблицы *cn* (681 строка) и последовательно присоединяя таблицы через индексные сканирования:



Ключевое свойство: соединение $mc \bowtie t$ выполняется через индексное сканирование t по первичному ключу ($t.id = mc.movie_id$, 0,52 строки на итерацию), а затем ci присоединяется также через индексное сканирование ($ci.movie_id = t.id$, 0,03 строки на итерацию). Промежуточные кардинальности: 681 → 4395 → 2270 → 76 → 56 → 52 → 52.

План декомпозиции на топологии (t_{plan} = 99 мс, t_{exec} = 2661 мс, деградация t_{exec} в 15,1 раза). Алгоритм распознавания разделяет граф на два фрагмента:



Фрагмент {*mc*, *t*} планируется отдельно. Поскольку ни *mc*, ни *t* не имеют достаточно селективных локальных фильтров, обе таблицы сканиру-

ются полностью (Seq Scan): $|mc| = 652\,282$ строки, $|t| = 253\,230$ строк после фильтра. Их хеш-соединение порождает промежуточный результат $|mc \bowtie t| = 207\,410$ строк.

Фрагмент $\{rt, ci, chn\}$ планируется без доступа к таблице t. Начиная с rt (1 строка), через индексное сканирование ci по роли получается 25 874 строки, затем chn фильтрует до 25 523.

На этапе сборки фрагменты соединяются хеш-соединением по условию $t.id = ci.movie_id$. Это соединение обрабатывает $207\,410 \times 25\,523$ кандидатов вместо 76 строк параметризованного индексного сканирования в плане DPsize. Именно этот узел порождает ~2,4 секунды из 2,66 секунд общего времени исполнения.

Вывод. Причина деградации — потеря параметризованного индексного пути $ci \rightarrow t$ при отделении t от ci в разные фрагменты. В плане DPsize таблица t доступна при построении путей для каждого подмножества, содержащего ci, и оптимизатор выбирает индексное сканирование t по первичному ключу. В плане декомпозиции таблица t уже законтрактирована в виртуальную вершину $\{mc, t\}$, и этот параметризованный путь недоступен.

Таблица t (title) является hub-вершиной в схеме IMDb — к ней присоединяются ci, mc, mk, mi, mi_idx, cc через внешние ключи. Аналогичная потеря параметризованных путей через t наблюдается в 78 из 81 деградировавших запросов. Подход с декомпозицией на топологии бесперспективен для схем с выраженными hub-вершинами, характерными для нормализованных реляционных баз данных. В схемах без доминирующих hub-вершин (например, в полностью денормализованных хранилищах) подход может давать иные результаты.

4.4. Подход 3. Динамическое программирование по подмножествам (DPsub)

Мотивация. Стандартный алгоритм DPsize перебирает подпланы по возрастанию размера $s = 2, 3, \dots, n$, на каждом уровне перебирая все разбиения $S = S_1 \cup S_2$ с $|S| = s$. Альтернативный алгоритм DPsub перечисляет подмножества в порядке возрастания числового значения битовой маски, что меняет последовательность, в которой кандидаты поступают в процедуру построения путей. Поскольку результат планирования в PostgreSQL зависит от порядка перебора (см. секцию «Архитектура планировщика PostgreSQL»), возникает вопрос: может ли альтернативный порядок обхода систематически улучшить t_{total}?

Описание алгоритма. Каждое подмножество отношений $S \subseteq \mathcal{R}$ кодируется битовой маской. Алгоритм поддерживает таблицу $BestPlan[S]$ и массив стоимостей $BestCost[S]$.

1. **Инициализация.** Для каждого базового отношения R_i задаётся $BestPlan[\{R_i\}] = R_i$, $BestCost[\{R_i\}] = \text{Cost}(R_i)$.

2. **Перебор подмножеств.** Для каждой битовой маски S в порядке возрастания ($S = 1, 2, \dots, 2^n - 1$) с $|S| \geqslant 2$ перебираются все нетривиальные разбиения $S = L \cup R$, $L \cap R = \varnothing$, $L < R$ (для устранения симметрии). Разбиения генерируются итерацией по подмаскам: $L = (-S) \,\&\, S$, затем $L = (L - S) \,\&\, S$ до $L = S$.

3. **Фильтрация.** Разбиение пропускается, если $BestPlan[L]$ или $BestPlan[R]$ не существует (подмножество не было достигнуто),

либо если L и R не связаны предикатом соединения.

4. **Построение и обновление.** Для допустимой пары (L, R) строится соединение вызовом процедуры построения путей. Если стоимость результата меньше $BestCost[S]$, запись обновляется.

Пространство поиска DPsub совпадает с пространством DPsize: оба алгоритма рассматривают все подмножества $S \subseteq \mathcal{R}$ и все их разбиения на пары. Асимптотическая сложность также совпадает: $O(3^n)$ разбиений в худшем случае, $O(2^n)$ памяти для таблицы подпланов.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan}: 0,95 (экономия 5%),
- t_{exec}: 1,05 (деградация 5%),
- t_{total}: 1,01 (нейтральный результат).

Из 113 запросов 77 показали улучшение t_{total}, однако суммарный результат нейтрален из-за нескольких сильных проигрышей. Экономия t_{plan} объясняется ранней фильтрацией несвязных пар (проверка предиката соединения выполняется до вызова процедуры построения путей). Деградация t_{exec} объясняется изменённым порядком перебора: подмножества обрабатываются по числовому значению маски, а не по размеру, поэтому кандидаты поступают в процедуру инкрементального отбора путей в другой последовательности. При ошибках оценок кардинальностей стоимости конкурирующих путей попадают в ε-окрестность, и изменённый порядок может как улучшить, так и ухудшить выбор — без систематического преимущества.

Вывод. Эксперимент с DPsub является контрольным: он показывает, что замена алгоритма перечисления при сохранении пространства поиска

не улучшает t_{total}. Проблема лежит не в порядке обхода подмножеств, а в двух других факторах:

- **Качество оценок кардинальностей** — определяет t_{exec}. Ни DPsize, ни DPsub не могут выбрать лучший план, если стоимостная модель ранжирует планы неверно из-за ошибок оценок.
- **Число вызовов процедуры построения путей** — определяет t_{plan}. Оба алгоритма перечисляют одно и то же множество пар, поэтому число вызовов совпадает с точностью до фильтрации несвязных пар.

Для улучшения t_{total} необходимо атаковать хотя бы один из этих факторов: либо уменьшить число вызовов (сокращение пространства поиска), либо повысить надёжность решений, принимаемых внутри пространства (фиксация соединений с предсказуемыми оценками). Этот вывод мотивирует подход с контракцией якорных зон, описанный далее.

4.5. Подход 4. DPhyp (реализация с открытым исходным кодом)

Мотивация. Алгоритм DPhyp [12] достигает нижней теоретической границы перебора среди DP-алгоритмов для произвольных графов соединений: он перечисляет *только* пары связных подграфов (csg-cmp-pairs), тогда как DPsize перебирает все подмножества, проверяя связность апостериори. Для разреженных графов (цепи, звёзды) это может дать экспоненциальное сокращение числа рассматриваемых пар. Целью эксперимента является проверка, приводит ли достижение нижней теоретической границы перебора к улучшению t_{total} в реальной СУБД.

Описание. Использована реализация DPhyp с открытым исходным кодом, интегрированная в PostgreSQL в виде расширения. Алгоритм строит гиперграф соединений и перечисляет пары связных подграфов (S_1, S_2) обходом по соседям, гарантируя, что каждая перечисленная пара связна и соединена хотя бы одним предикатом. Для каждой пары вызывается процедура построения путей, аналогичная стандартному DPsize.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan}: 1,015 (деградация 1,5%),
- t_{exec}: 1,131 (деградация 13,1%),
- t_{total}: 1,088 (деградация 8,8%).

Алгоритм, достигающий нижней теоретической границы перебора, показал результат *хуже* стандартного DPsize по всем трём метрикам.

Анализ причин.

1. Теоретическое преимущество не реализуется на практике. Запросы JOB содержат от 5 до 17 таблиц. При таких размерах число связных подграфов #csg(G) составляет от десятков до нескольких тысяч — значения, при которых и DPsize, и DPhyp завершаются быстро. Экономия от перечисления только связных подграфов проявляется при $n \gtrsim 20$, где #csg начинает существенно отличаться от 2^n. Более того, свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на само перечисление пар. DPhyp и DPsize вызывают эту процедуру для одних и тех же допустимых пар, различаясь лишь порядком вызовов.

2. Изменённый порядок перечисления ухудшает t_{exec}. Это центральная причина деградации. DPhyp перечисляет пары обходом по соседям ги-

перграфа, а DPsize — в порядке возрастания размера подмножества. Как показано в секции «Архитектура планировщика PostgreSQL», при ошибках оценок кардинальностей стоимости конкурирующих путей попадают в ε-окрестность, и путь, поступивший в процедуру инкрементального отбора раньше, получает структурное преимущество. DPsize на каждом уровне сначала обрабатывает левосвязанные разбиения, порождая планы, для которых стоимостная модель PostgreSQL наиболее точна (меньше промежуточных соединений — меньше каскадных ошибок). DPhyp не имеет этого свойства: порядок перечисления определяется структурой гиперграфа и может отдавать приоритет ветвистым разбиениям с менее надёжными оценками.

3. Неполная конструкция гиперграфа. Реализация с открытым исходным кодом не строит обобщённые гиперрёбра (generalized hyperedges), необходимые для корректной обработки внешних соединений и сложных предикатов, а также не реализует полноценное обнаружение конфликтов (CD-C, conflict detection for commutativity). Это может приводить к пропуску допустимых разбиений или к построению планов, которые стандартный DPsize нашёл бы через другие пути перечисления.

Вывод. Эксперименты с DPsub и DPhyp показывают, что замена алгоритма перечисления при сохранении пространства поиска и процедуры построения путей не даёт систематического улучшения t_{total}. Это согласуется с механизмом ε-зависимости от порядка перебора: при ошибочных оценках кардинальностей изменение порядка может как улучшить, так и ухудшить отдельные запросы, без устойчивого преимущества.

Более того, если новый порядок перечисления систематически смещает приоритет в сторону разбиений с менее надёжными оценками кардинально-

стей, результат может стать хуже. Нижняя теоретическая граница перебора является свойством алгоритма перечисления, но t_{total} определяется качеством выбранного плана и числом вызовов процедуры построения путей — ни то, ни другое DPhyp не улучшает для запросов с 5–17 таблицами. Следует подчеркнуть, что результат характеризует данную реализацию DPhyp, а не алгоритм в целом. Полноценная реализация с корректным построением обобщённых гиперрёбер и обнаружением конфликтов может показать иные результаты. Тем не менее пункты 1 (теоретическое преимущество не реализуется при $n \leqslant 17$) и 2 (зависимость от порядка перечисления) являются свойствами архитектуры PostgreSQL, а не конкретной реализации DPhyp.

4.6. Подход 5. Модифицированный DPhyp (DPhypMod)

Мотивация. Эксперимент с DPhyp (подход 5) выявил три конкретные причины деградации: нарушение поуровневого инварианта PostgreSQL, невыгодный порядок обработки кандидатов и избыточные вызовы процедуры построения путей на промежуточных уровнях. Модифицированный DPhyp адресует каждую из этих причин отдельной модификацией, сохраняя оригинальный алгоритм перечисления csg-cmp-пар без изменений.

Модификация 1. Поуровневая материализация. Стандартный DPhyp строит отношения в порядке обхода гиперграфа, произвольно чередуя подпланы разных уровней. Однако архитектура планировщика PostgreSQL опирается на инвариант: все отношения уровня k должны быть полностью построены и финализированы до начала построения отношений уровня $k+1$. Нарушение этого инварианта приводит к неполному распространению

классов эквивалентности и пропуску путей.

Модификация разделяет работу DPhyp на две фазы:

1. **Перечисление.** Оригинальный алгоритм DPhyp перечисляет все csg-cmp-пары, но не вызывает процедуру построения путей. Вместо этого для каждой пары (S_1, S_2) записывается список кандидатов в соответствующий узел DP-таблицы и вычисляется облегчённая оценка кардинальности $|\widehat{S_1 \bowtie S_2}|$.

2. **Материализация.** Узлы DP-таблицы группируются по уровню (числу базовых отношений) и обрабатываются строго последовательно: сначала все узлы уровня 2, затем уровня 3, и т. д. Для каждого узла вызывается процедура построения путей по всем его кандидатам, после чего выполняется финализация. Это восстанавливает поуровневый инвариант PostgreSQL.

Модификация 2. Приоритет левосвязанных разбиений. Как показано в секции «Архитектура планировщика PostgreSQL», стандартный DPsize на каждом уровне обрабатывает левосвязанные разбиения ($|S_1| = s - 1, |S_2| = 1$) до ветвистых. Это обеспечивает приоритет параметризованных путей (вложенный цикл с индексным сканированием) в процедуре инкрементального отбора. В DPhyp порядок кандидатов определяется обходом гиперграфа и не имеет этого свойства.

Модификация вводит два уровня сортировки:

1. **Между узлами одного уровня.** Узлы уровня k группируются по значению μ — минимальному размеру дочернего подплана среди всех кандидатов узла. Узлы с $\mu = 1$ (левосвязанные) обрабатываются первыми, затем $\mu = 2$, и т. д.

2. **Внутри списка кандидатов одного узла.** Пары кандидатов сортируются по $\min(|S_1|, |S_2|)$ по возрастанию, так что левосвязанные разбиения узла поступают в процедуру построения путей раньше ветвистых.

Обе сортировки воспроизводят порядок обработки стандартного `join_search_one_level` в контексте DPhyp-перечисления.

Модификация 3. Отсечение на промежуточных уровнях по кардинальности. Распределение числа узлов DP-таблицы по уровням имеет характерную форму нормального распределения с максимумом на средних уровнях ($s \approx n/2$). Для запроса с $n = 11$ таблицами на уровнях 5–8 сосредоточено ~86% всех узлов. Основная вычислительная нагрузка планирования приходится именно на эти уровни.

На каждом уровне из области горба ($\lceil n/3 \rceil + 1 \leqslant s \leqslant n - \lfloor n/3 \rfloor$) выполняется отсечение узлов с аномально высокой оценкой кардинальности:

1. Вычисляется $\min_S \widehat{|S|}$ среди всех узлов уровня s.
2. Узлы с $\widehat{|S|} > \alpha \cdot \min_S \widehat{|S|}$ отсекаются (в реализации $\alpha = 300$).
3. Дополнительно отсекаются узлы с $\widehat{|S|} > C_{\text{out}}^{\text{GOO}}$, где $C_{\text{out}}^{\text{GOO}}$ — сумма кардинальностей промежуточных результатов жадного плана, вычисленная облегчённым GOO по оценкам $\widehat{|S|}$ из фазы перечисления. Если кардинальность одного промежуточного результата превышает суммарный объём всех промежуточных результатов жадного плана, такой подплан заведомо неоптимален.
4. **Гарантия покрытия.** После отсечения проверяется, что каждое базовое отношение $R_i \in \mathcal{R}$ входит хотя бы в один выживший узел уровня s. Если отношение потеряно, восстанавливается узел с ми-

нимальной $|\widehat{S}|$ среди отсечённых, содержащий это отношение. Это предотвращает отсечение всех путей через критически важные базовые отношения.

На крайних уровнях ($s \leqslant \lceil n/3 \rceil$ и $s > n - \lfloor n/3 \rfloor$) отсечение не выполняется: число узлов мало, стоимостная модель ещё точна, а разнообразие путей критически важно для качества финального плана.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan}: 0,966 (экономия 3,4%),
- t_{exec}: 0,967 (улучшение 3,3%),
- t_{total}: 0,967 (улучшение 3,3%).

Из 113 запросов 99 показали улучшение t_{total}, что значительно лучше, чем исходный DPhyp (78 из 113) и DPsub (77 из 113). В сравнении с DPhyp модификация улучшает результат в 98 запросах из 113: поуровневая материализация и приоритет левосвязанных разбиений устраняют основные причины деградации исходного алгоритма.

Абсолютные суммарные времена (по 113 запросам):

- DPsize: $t_{\text{plan}} = 52{,}3$ с, $t_{\text{exec}} = 88{,}8$ с, $t_{\text{total}} = 141{,}2$ с.
- DPhypMod: $t_{\text{plan}} = 50{,}5$ с, $t_{\text{exec}} = 85{,}9$ с, $t_{\text{total}} = 136{,}5$ с.
- DPhyp: $t_{\text{plan}} = 53{,}1$ с, $t_{\text{exec}} = 100{,}5$ с, $t_{\text{total}} = 153{,}6$ с.

Анализ: почему улучшение ограничено 3,3%.

1. Пространство поиска совпадает с DPsize. Все три модификации влияют на *порядок обработки* и *количество* узлов DP-таблицы, но не расширяют и не сужают множество рассматриваемых разбиений по сравнению с DPsize. Поуровневая материализация и сортировка кандидатов восстанав-

ливают инварианты стандартного планировщика — но не превосходят их. DPhypMod *приближается* к DPsize по качеству планов, а не улучшает его.

2. Отсечение экономит мало. Запросы JOB содержат от 5 до 17 таблиц. При таких размерах число узлов DP-таблицы составляет от десятков до нескольких тысяч, а горб распределения — область, где отсечение действует — содержит от десятков до нескольких сотен узлов. Отсечение по кардинальности с консервативным порогом $\alpha = 300$ и гарантией покрытия удаляет лишь единицы узлов на каждом уровне. Экономия t_{plan} от сокращения вызовов процедуры построения путей составляет 3,4%, что соответствует удалению ~3% узлов. Более агрессивное отсечение ($\alpha < 100$) рискует удалить узлы, через которые проходит оптимальный план, поскольку оценки кардинальностей на промежуточных уровнях ненадёжны.

3. Процедура построения путей остаётся узким местом. Свыше 90% времени t_{plan} приходится на процедуру построения путей, а не на перечисление или сортировку узлов. Замена алгоритма перечисления (DPsize на DPhyp) и оптимизация порядка обработки не могут существенно сократить число вызовов этой процедуры, потому что число допустимых разбиений определяется топологией графа соединений, а не алгоритмом их обнаружения.

4. Потолок для DPhyp-подхода на JOB. Результат DPhypMod — 3,3% улучшения — представляет собой верхнюю границу того, что достижимо модификациями DPhyp при сохранении стандартной процедуры построения путей и стоимостной модели PostgreSQL на запросах JOB с 5–17 таблицами. Для существенного улучшения необходимо воздействовать не на алгоритм перечисления, а на само пространство поиска — уменьшая число

отношений, подаваемых на вход DP, путём предварительной контракции подграфов с предсказуемыми свойствами. Этот вывод мотивирует переход к подходу с контракцией якорных зон.

4.7. Подход 6. Контракция якорных зон

Мотивация. Эксперименты с подходами 3–5 показали, что замена алгоритма перечисления при сохранении пространства поиска не улучшает t_{total}. Потолок DPhypMod — 3,3% — определяется тем, что число вызовов процедуры построения путей (доминирующей во времени планирования) задано топологией графа и не сокращается перестановкой порядка обхода. Эксперимент с декомпозицией на топологии (подход 2) показал, что разбиение графа на фрагменты может сократить t_{plan}, но при неудачной фрагментации разрушаются параметризованные пути через hub-вершины, катастрофически ухудшая t_{exec}.

Отсюда следует стратегия: сокращать пространство поиска, но делать это *безопасно* — контрактируя только те подграфы, для которых оценки кардинальностей заведомо надёжны, а hub-вершины не могут быть изолированы от своих соседей.

Определения. Пусть задан граф соединений $G = (\mathcal{R}, \mathcal{E})$ с n вершинами.

- **Якорная вершина** — вершина $v \in \mathcal{R}$, удовлетворяющая двум условиям:

  1. $|v| < R_{\max}$ (абсолютный порог; в реализации $R_{\max} = 50\,000$),
  2. $|v| \cdot \alpha < \max_{u \in N(v)} |u|$ (контрастное условие; $\alpha = 100$).

  Якорь — это малая таблица, имеющая хотя бы одного соседа, который

в α раз больше. Интуиция: соединение якоря с крупным соседом высокоселективно, а его кардинальность оценивается надёжно, поскольку на первом шаге ошибки ещё не каскадировались.

- **Якорная зона** — связная компонента подграфа, индуцированного на множестве вершин $\{v\} \cup N(v)$ для всех якорей v. Зона включает якорь и всех его непосредственных соседей (1-hop).

Алгоритм. Входом является список базовых отношений и граф соединений. Алгоритм реализует каскад из трёх стратегий с убывающей точностью:

Шаг 1. Построение графа и оценка сложности. Строится граф соединений G и вычисляется $\#\mathrm{csg}(G)$ — число связных подграфов, определяющее объём DP-таблицы. Если $\#\mathrm{csg}(G) \leqslant \tau$ (в реализации $\tau = 350$), граф передаётся стандартному DPsize без модификаций: при малом #csg точный перебор быстр и даёт оптимальный план.

Шаг 2. Контракция якорных зон. Если $\#\mathrm{csg}(G) > \tau$, выполняется контракция:

1. Маркировка. Для каждой вершины проверяется якорное условие. Все якори и их 1-hop-соседи маркируются.
2. Выделение зон. Маркированные вершины разбиваются на связные компоненты обходом в ширину (BFS) по подграфу маркированных вершин. Каждая связная компонента образует одну зону.
3. Планирование зоны. Каждая зона планируется *стандартным DPsize*, поскольку зоны малы (типично 2–5 вершин).
4. Проверка. Если кардинальность результата планирования зоны превышает порог L (в реализации $L = 100\,000$ строк), контракция зоны

отменяется: вершины возвращаются в граф как отдельные базовые отношения.

5. Перестроение графа. Успешно спланированные зоны заменяются виртуальными вершинами. Строится новый граф соединений и пересчитывается #csg.

После контракции #csg пересчитывается. Если #csg $\leqslant \tau$, новый граф передаётся DPsize.

Шаг 3. Жадный алгоритм (fallback). Если после контракции #csg $> \tau$, оставшийся граф планируется комбинированным жадным алгоритмом $\mathrm{GOO}_{\mathrm{comb}}$ (подход 1, вариант 3).

Если граф имеет несколько компонент связности, они объединяются тем же $\mathrm{GOO}_{\mathrm{comb}}$ в режиме декартова произведения.

Выбор параметров. Подход использует четыре параметра: $\tau = 350$, $R_{\max} = 50\,000$, $\alpha = 100$, $L = 100\,000$.

Порог $\tau = 350$ определяет границу переключения между точным DP и каскадом с контракцией. Значение выбрано из условия, что стандартный DPsize обрабатывает 350 связных подграфов за время, сопоставимое с накладными расходами на построение графа соединений и контракцию (~5–10 мс). Для цепи из n вершин #csg $= O(n^2)$: цепь из 18 таблиц имеет #csg ≈ 170, что ниже порога; звезда из 10 вершин имеет #csg ≈ 512, что выше. Таким образом, $\tau = 350$ разделяет «лёгкие» и «тяжёлые» графы на запросах с 5–17 таблицами.

Значение $R_{\max} = 50\,000$ отсекает все таблицы, сопоставимые по размеру с fact-таблицами типичных аналитических схем (10^5–10^7 строк). Контрастный множитель $\alpha = 100$ требует двухпорядкового различия между якорем

и его крупнейшим соседом, ограничивая якори справочниками и малыми dimension-таблицами, для которых соединения высокоселективны. Порог контракции $L = 100\,000$ обеспечивает, что виртуальная вершина не вносит в граф отношение с кардинальностью, характерной для fact-таблиц.

Все четыре значения подобраны экспериментально на JOB и не валидировались на других бенчмарках. Однако выбранные значения консервативны: уменьшение $R_{\max}$ или увеличение α сужают множество якорей (меньше контракций, ближе к DPsize), а не расширяют его. Деградация при переносе на другую схему маловероятна — скорее уменьшится выигрыш.

Свойство безопасности: защита hub-вершин. Критическое свойство алгоритма: якорное условие $|v| < 50\,000 \;\wedge\; |v| \cdot 100 < \max_{u \in N(v)} |u|$ гарантирует, что hub-вершины (в IMDb — таблица title с $|t| = 2{,}5 \cdot 10^6$) *никогда* не становятся якорями. Hub-вершина может попасть в якорную зону только как сосед якоря — но в этом случае она входит в зону *вместе со всеми* соседями якоря, а не изолируется от части своих соседей. Это исключает ситуацию, разрушавшую подход 2: потерю параметризованных путей через hub-вершину.

Более того, в типичных запросах JOB hub-вершина имеет степень, превышающую число вершин в якорной зоне. Поскольку зона включает только 1-hop-соседей якоря (а не hub), hub-вершина остаётся связанной с незатронутыми вершинами и сохраняет все параметризованные пути при последующем DPsize.

Свойство надёжности оценок. Якорные зоны содержат 2–5 вершин, из которых как минимум одна (якорь) имеет $|v| < 50\,000$ строк. Соединения внутри зоны — это первые 1–3 шага планирования, где мультипликатив-

ное каскадирование ошибок кардинальностей ещё не успело накопиться. Стандартный DPsize внутри зоны выбирает оптимальный план с высокой вероятностью, поскольку стоимостная модель надёжна для малых подпланов (см. секцию «Оценки в модели стоимости»). Проверка $|zone_plan| \leqslant L$ является дополнительной защитой: если результат зоны оказался неожиданно большим (признак ошибки оценки), контракция отменяется.

Результаты. Суммарные отношения к стандартному DPsize:

- t_{plan}: 0,778 (экономия 22,2%),
- t_{exec}: 0,889 (улучшение 11,1%),
- t_{total}: 0,848 (улучшение 15,2%).

Из 113 запросов 102 показали улучшение t_{total}, 11 — деградацию. Планирование улучшилось в 100 запросах, исполнение — в 98. Оба показателя улучшились одновременно в 89 запросах.

Абсолютные суммарные времена:

- DPsize: $t_{\text{plan}} = 52{,}3$ с, $t_{\text{exec}} = 88{,}8$ с, $t_{\text{total}} = 141{,}2$ с.
- Контракция якорных зон: $t_{\text{plan}} = 40{,}7$ с, $t_{\text{exec}} = 79{,}0$ с, $t_{\text{total}} = 119{,}7$ с.

Суммарная экономия — 21,5 секунды на 113 запросах, из которых 11,6 с приходится на t_{plan} и 9,9 с — на t_{exec}.

Анализ улучшения t_{plan}. Контракция якорных зон уменьшает число вершин графа соединений, подаваемого на вход DPsize. Поскольку #csg(G) растёт экспоненциально с числом вершин, удаление даже 2–3 вершин может сократить #csg в несколько раз. Наибольшая экономия наблюдается на запросах с 13–17 таблицами:

- Запрос 33a (14 таблиц): t_{plan} уменьшилось с 627 мс до 98 мс (в 6,4

раза). Контракция сократила число вершин на 5–6, уменьшив #csg с нескольких тысяч до значения ниже порога $\tau = 350$.

- Запрос 29a (17 таблиц): t_{plan} уменьшилось с 2608 мс до 1292 мс (в 2,0 раза).
- Запрос 26a (12 таблиц): t_{plan} уменьшилось с 759 мс до 352 мс (в 2,2 раза).

Для запросов с $\leqslant 10$ таблицами #csg уже невелик, и экономия t_{plan} составляет единицы процентов.

Анализ улучшения t_{exec}. Улучшение t_{exec} на 11,1% не является целью подхода, но является его следствием. Контракция фиксирует высокоселективные соединения (якорь + соседи) до основного DP. Внутри зоны стандартный DPsize находит оптимальный план, поскольку зоны малы и оценки кардинальностей для первых соединений надёжны. Виртуальная вершина, созданная контракцией, несёт точную оценку кардинальности (результат DPsize на зоне) и фиксирует выбор физического пути (например, параметризованное индексное сканирование внутри зоны). Это решение *не подвергается пересмотру* на последующих уровнях DPsize, где ошибки каскадируются и стоимостная модель менее надёжна. Фактически контракция изолирует наиболее предсказуемые решения от ε-шума на средних уровнях DP.

Анализ деградаций. Из 11 проигрышей 10 имеют деградацию менее 12%, а суммарные абсолютные потери составляют 1,8 секунды (против 23,3 секунды суммарного выигрыша). Наибольшая деградация — запрос 17a (t_{total}: 5098 → 6394 мс, +26,8%): t_{plan} увеличился с 278 до 354 мс, t_{exec} — с

4821 до 6040 мс. Причина: контракция выделила зону, которая в данном запросе не содержит наиболее селективного пути; виртуальная вершина ограничивает DPsize в выборе альтернативного порядка, который оказался бы лучше. Тем не менее деградация умеренна и ограничена гарантией покрытия и проверкой $|zone_plan| \leqslant L$.

Среди 13 запросов с деградацией t_{plan} (отношение $>$ 1,0) причиной является накладной расход на построение графа соединений, вычисление #csg и планирование зон, который не окупается на запросах с #csg $\leqslant \tau$, где стандартный DPsize и так быстр. В таких случаях каскад корректно маршрутизирует запрос напрямую в DPsize (шаг 1), но накладные расходы на проверку условия #csg $\leqslant \tau$ составляют 5–10% от t_{plan}.

Сравнение с другими подходами. Контракция якорных зон — единственный подход из шести протестированных, который одновременно улучшает и t_{plan}, и t_{exec} относительно DPsize. Для сравнения:

- GOO_{comb} (подход 1): $t_{\text{plan}} = 0{,}81$, но $t_{\text{exec}} = 1{,}19$ — экономия планирования перекрывается деградацией исполнения.
- DPhypMod (подход 4): $t_{\text{total}} = 0{,}967$ — улучшение 3,3%, в 4,6 раза меньше, чем у якорного подхода.
- DPsub (подход 3): нейтральный результат ($t_{\text{total}} = 1{,}01$).
- Декомпозиция (подход 2): деградация в 2,5 раза.

Результат объясняется тем, что якорный подход — единственный, который одновременно сокращает число вызовов процедуры построения путей (уменьшая t_{plan}) *и* фиксирует надёжные решения до основного DP (улучшая t_{exec}), не разрушая при этом параметризованные пути через hub-вершины.

5. Ограничения и применимость результатов

Экспериментальные результаты получены в условиях, которые необходимо учитывать при оценке их обобщаемости.

Один бенчмарк, одна схема. Все эксперименты выполнены на Join Order Benchmark (113 запросов к базе IMDb). Схема IMDb имеет выраженную hub-вершину — таблицу title ($2{,}5 \cdot 10^6$ строк), к которой через внешние ключи присоединяются большинство fact-таблиц. Якорный подход эксплуатирует эту структуру: малые справочники рядом с крупными fact-таблицами порождают высококонтрастных якорей. В схемах без выраженных hub-вершин (денормализованные хранилища, графовые базы данных) множество якорей может оказаться пустым, и подход сведётся к стандартному DPsize без улучшения. В схемах с несколькими hub-вершинами сопоставимого размера подход остаётся применимым, но выигрыш не валидирован.

Прогретый кэш. Все данные загружены в буферный кэш СУБД, что исключает дисковый ввод-вывод. В реальных системах часть данных считывается с диска, и стоимостная модель PostgreSQL учитывает соотношение дискового и кэшированного доступа. Прогрев кэша убирает эту нелинейность, что делает сравнение чище, но может завышать различия между планами: при дисковом вводе-выводе разница в числе обработанных строк частично маскируется латентностью диска.

Одна аппаратная конфигурация. Эксперименты выполнены на одной машине (Intel Core Ultra 7 258V, 32 ГБ RAM). Абсолютные значения t_{plan} и t_{exec} зависят от скорости процессора и объёма памяти. Однако *отношения*

времён (нормализованные результаты) определяются алгоритмическими свойствами планировщика, а не аппаратной конфигурацией, и должны воспроизводиться на другом оборудовании.

Параллелизм. Конфигурация разрешает параллельное исполнение операторов (`max_parallel_workers = 8`). PostgreSQL использует параллелизм избирательно: не все планы параллелизуются, и выбор параллельного плана зависит от оценки стоимости. Изменение порядка соединений может косвенно влиять на решение о параллелизации, что добавляет неконтролируемый фактор в t_{exec}.

Диапазон размеров запросов. Запросы JOB содержат от 5 до 17 таблиц. Для запросов с $n \leqslant 10$ стандартный DPsize и так быстр ($t_{\text{plan}} < 100$ мс), и якорный подход даёт минимальный выигрыш. Для запросов с $n > 20$ поведение подхода не исследовано; в частности, при большом числе якорей контракция может создать граф, в котором несколько виртуальных вершин с неточными оценками кардинальностей взаимодействуют с основным DP, что может привести к каскадированию ошибок.

Список литературы

1. Leis V. и др. How good are query optimizers, really? // Proc. VLDB Endow. 2015. Т. 9, № 3. С. 204–215.
2. Ibaraki T., Kameda T. On the optimal nesting order for computing N-relational joins // ACM Trans. Database Syst. 1984. Т. 9, № 3. С. 482–502.
3. Karp R.M. Reducibility among combinatorial problems // Complexity of computer computations. Plenum Press, 1972. С. 85–103.

4. Moerkotte G., Fender P., Eich M. On the correct and complete enumeration of the core search space // Proceedings of the 2013 ACM SIGMOD international conference on management of data. 2013.
5. Fegaras L. A new heuristic for optimizing large queries // Proceedings of the 9th International Conference on Database and Expert Systems Applications (DEXA). 1998.
6. Neumann T. Query optimization. Technical University of Munich, 2025.
7. Moerkotte G., Neumann T. Analysis of two existing and one new dynamic programming algorithm for the generation of optimal bushy join trees without cross products // Proceedings of the 32nd International Conference on Very Large Data Bases (VLDB). 2006.
8. Moerkotte G., Neumann T. Dynamic programming strikes back // Proceedings of the 2008 ACM SIGMOD international conference on management of data. 2008.
9. Bennett K., Ferris M.C., Ioannidis Y.E. A genetic algorithm for database query optimization // Proceedings of the 4th International Conference on Genetic Algorithms. 1991.
10. Allam J.R. Evaluation of a greedy join-order optimization approach using the IMDB dataset. University of Magdeburg, 2018.
11. Neumann T., Radke B. Adaptive optimization of very large join queries // Proceedings of the 2018 international conference on management of data (SIGMOD). 2018.
12. Neumann T., Moerkotte G. An algorithm for producing all minimal connected subgraph partitions of a hypergraph // Proceedings of the 2006 international conference on management of data. 2006.